

08.OKTOBER.2026

Ausbildungsbetrieb: Timewaste AG

Projektbetreuer: Stephen Vincent Strange

wasgehtsiedasan@bikini-bottom.com – 0900 666 666

Musteradresse 8, 11111 Musterstadt

IHK Probeprüfung

2026

Ausbildungsberuf:

Fachinformatiker - Anwendungsentwicklung

DIGITALE WERKZEUGAUSLEIHE

ENTWICKLUNG EINER DESKTOPANWENDUNG ZU VERWALTUNG VON
WERKZEUG- UND GERÄTEAUSLEIHEN

PRÜFLING: LANG, ALEXANDER

MUSTERADRESSE 2, 22222 AUCHMUSTERSTADT – GEB; 10.07.1994
xXshadowmasterXx@hotmail.com - 0160 janzeheim

Digitale Werkzeugausleihe

Entwicklung einer Desktopanwendung zur Verwaltung von Werkzeug- und Geräteausleihen

Dokumentation zur betrieblichen Projektarbeit

IHK Probeprüfung 2026, Fachinformatiker für Anwendungsentwicklung

Prüfungsbezirk: IHK Region Stuttgart

Prüfling: Alexander Lang, Identnummer 555, Prüflingsnummer 42069

Ausbildungsbetrieb: Timewaste AG

Projektbetreuer: Stephen Vincent Strange

Projektzeitraum laut Antrag: 11.09.2026 bis 29.09.2026

Tatsächlicher Projektzeitraum: 10.09.2026 bis 08.10.2026

Abgabe: 08.10.2026

Inhaltsverzeichnis

Digitale Werkzeugausleihe	1
Inhaltsverzeichnis	2
Abbildungsverzeichnis.....	4
Tabellenverzeichnis.....	4
Abkürzungsverzeichnis und Glossar	6
1 Projektbeschreibung.....	8
1.1 Projektumfeld	8
1.2 Auftraggeber und Anforderung	8
1.3 Ist-Zustand.....	8
1.4 Soll-Zustand und Projektziel	8
1.5 Projektabgrenzung	8
1.6 Teilaufgaben	9
2 Projektplanung	10
2.1 Personalplanung	10
2.2 Sachmittelplanung	10
2.3 Kostenplanung	10
2.4 Terminplanung	11
2.5 Vorgehensmodell	12
2.6 Projektablauf.....	12
3 Analysephase	13
3.1 Make-or-Buy-Entscheidung	13
3.2 Wirtschaftlichkeit und Amortisation	14
3.3 Anwendungsfälle.....	14
3.4 Anforderungen und Geschäftsregeln	14
3.5 Qualitätsanforderungen	15
4 Entwurfsphase	15
4.1 Zielplattform	15
4.2 Architektur	16
4.3 Datenmodell	16
4.4 Entwurfsentscheidungen	17
4.5 Benutzeroberfläche	18

4.6 Maßnahmen zur Qualitätssicherung	18
5 Implementierungsphase	18
5.1 Datenhaltung	18
5.2 Datenimport.....	18
5.3 Geschäftslogik	18
5.4 Benutzeroberfläche	19
5.5 Export	20
5.6 Vorbereitete Funktionen ohne Oberfläche	20
6 Qualitätssicherung und Abnahme	21
6.1 Testkonzept	21
6.2 Geplante Unit-Tests	21
6.3 Abnahmetests.....	21
6.4 Abnahme durch den Auftraggeber	22
7 Einführung und Dokumentation.....	22
7.1 Einführung	22
7.2 Dokumentation	23
8 Projektabschluss	23
8.1 Soll-Ist-Vergleich.....	23
8.2 Abweichungen vom Entwurf.....	24
8.3 Lessons Learned	24
8.4 Fazit.....	24
8.5 Ausblick.....	25
Anhang	25
A1 Projektantrag	25
A2 Zeitplanung mit Ist-Werten	25
A3 Klassendiagramm	26
A4 Screenshots der Anwendung	28
A5 Anwenderdokumentation	28
A6 Codeauszüge	32
A7 Testprotokoll.....	35
Quellenverzeichnis.....	35
Hinweis zum Einsatz von KI-Werkzeugen	36
Eidesstattliche Erklärung	37

Abbildungsverzeichnis

- Abb. 1: Projektablauf als Gantt-Diagramm (Abschnitt 2.6)
- Abb. 2: Anwendungsfalldiagramm (Abschnitt 3.3)
- Abb. 3: Schichtenarchitektur (Abschnitt 4.2)
- Abb. 4: Entity-Relationship-Modell der Datenbank (Abschnitt 4.3)
- Abb. 5: Ablauf einer Ausleihe (Abschnitt 5.3)
- Abb. 6: Klassendiagramm Teil 1, Fachlogik und Datenmodell (Anhang A3)
- Abb. 7: Klassendiagramm Teil 2, Datenschicht (Anhang A3)
- Abb. 8: Klassendiagramm Teil 3, Oberflächenlogik (Anhang A3)
- Abb. 9: Hauptfenster nach dem Programmstart (Anhang A5)
- Abb. 10: Ausgefüllter Ausleihdialog mit englischem Datumsformat (Anhang A5)
- Abb. 11: Filter „Offene Verleihe“ (Anhang A5)
- Abb. 12: Filter „Überfällige Verleihe“ (Anhang A5)
- Abb. 13: Speicherort für den Export wählen, Dialog in englischer Systemsprache (Anhang A5)
- Abb. 14: Exportierte Überfälligkeitsliste in einer englischen Excel-Version (Anhang A5)
- Abb. 15: Prüfungen vor einer Ausleihe in LendService.BorrowItem (Anhang A6)
- Abb. 16: Prüfungen vor einer Rückgabe in LendService.ReturnItem (Anhang A6)
- Abb. 17: Ergebnisobjekt TransactionResult (Anhang A6)
- Abb. 18: Methode Refresh im MainViewModel (Anhang A6)
- Abb. 19: Zusammenfassung der KI zur Kürzung der Dokumentation (Hinweis zum Einsatz von KI-Werkzeugen)

Tabellenverzeichnis

- Tab. 1: Projektkosten (Abschnitt 2.3)
- Tab. 2: Terminplanung mit Soll- und Ist-Werten (Abschnitt 2.4)
- Tab. 3: Vergleich der Lösungswege (Abschnitt 3.1)
- Tab. 4: Abnahmetests (Abschnitt 6.3)

Tab. 5: Umsetzung der Anforderungen (Abschnitt 8.1)

Abkürzungsverzeichnis und Glossar

.NET: Software-Plattform von Microsoft für die Anwendungsentwicklung. Die Anwendung läuft auf Version 10.

Amortisationszeit: Zeitraum, nach dem die Einsparungen eines Projekts dessen Kosten ausgeglichen haben.

C#: Objektorientierte Programmiersprache von Microsoft auf Basis von .NET.

Command: Objekt, das eine Aktion der Oberfläche, etwa einen Buttonklick, mit einer Methode im ViewModel verbindet. Es legt auch fest, ob die Aktion gerade ausführbar ist.

CSV: Comma-Separated Values, ein Textformat für Tabellendaten. In diesem Projekt mit Semikolon als Trennzeichen, das ein deutsch eingestelltes Excel direkt als Spaltentrenner erkennt.

Data Binding: Verknüpfung eines Oberflächenelements mit einer Eigenschaft im ViewModel. Ändert sich der Wert, aktualisiert sich die Anzeige automatisch.

DbContext: Zentrale Klasse von EF Core, die die Verbindung zur Datenbank hält und Änderungen speichert. Im Projekt heißt sie LendContext.

EF Core: Entity Framework Core, ein objektrelationaler Mapper von Microsoft. Er bildet C#-Klassen auf Datenbanktabellen ab.

Enum: Aufzählungstyp mit festen Werten. Der Gerätestatus ist als Enum ItemStatus umgesetzt.

ERM: Entity-Relationship-Modell, die grafische Darstellung von Tabellen und ihren Beziehungen.

Fremdschlüssel: Spalte, die auf den Primärschlüssel einer anderen Tabelle verweist, etwa von der Ausleihe auf das Gerät.

Git: Versionsverwaltung, die jeden Arbeitsstand als Commit speichert und frühere Stände wiederherstellbar macht.

Interface: Schnittstelle, die festlegt, welche Methoden eine Klasse anbieten muss, ohne deren Umsetzung vorzugeben.

Inventarnummer: Eindeutige Nummer, unter der ein Gerät im Lager geführt wird.

Kladde: Handschriftlich geführtes Ausleihbuch, das bisher am Lagerregal lag.

Konstruktorinjektion: Eine Klasse erhält die Objekte, die sie benötigt, über ihren Konstruktor, statt sie selbst zu erzeugen. Dadurch lassen sich diese Objekte im Test austauschen.

LINQ: Language Integrated Query, eine Abfragesprache in C# zum Filtern, Gruppieren und Sortieren von Daten. EF Core übersetzt LINQ-Abfragen in SQL.

Make-or-Buy: Entscheidung, ob eine Lösung selbst entwickelt oder fertig eingekauft wird.

Migration: Von EF Core erzeugte Code-Datei, die eine Änderung am Datenbankschema beschreibt und nachvollziehbar macht.

MVVM: Model-View-ViewModel, ein Architekturmuster für WPF, das Oberfläche und Logik trennt.

Repository: Klasse, die den Datenbankzugriff für eine Tabelle kapselt. Die übrigen Schichten kennen nur ihr Interface.

Service-Schicht: Schicht mit der Fachlogik. Hier prüft der LendService die Geschäftsregeln vor jeder Buchung.

SQLite: Dateibasierte Datenbank ohne eigenen Server. Die gesamte Datenbank liegt in einer Datei neben der Anwendung.

UML: Unified Modeling Language, eine standardisierte Notation für Diagramme in der Softwareentwicklung.

Unit-Test: Automatisierter Test, der eine einzelne Methode isoliert prüft.

UTF-8 und BOM: UTF-8 ist eine verbreitete Zeichenkodierung. Die Byte Order Mark am Dateianfang sorgt dafür, dass Excel Umlaute korrekt erkennt.

WPF: Windows Presentation Foundation, das Oberflächen-Framework von Microsoft für Windows-Desktopanwendungen.

XAML: XML-basierte Beschreibungssprache, in der WPF-Oberflächen aufgebaut werden.

1 Projektbeschreibung

Ziel des Projekts ist eine WPF-Desktopanwendung, die die handschriftliche Ausleihkladde im Lager der Timewaste AG ablöst. Sie zeigt jederzeit, welches Gerät verfügbar, verliehen oder überfällig ist.

1.1 Projektumfeld

Die Timewaste AG ist ein Handwerksbetrieb mit 45 Mitarbeitern, davon 28 Monteure im Außendienst. Im Zentrallager geben zwei Lageristen rund 120 Werkzeuge und Messgeräte im Gesamtwert von etwa 95.000 Euro an die Monteure aus, etwa Isolationsmessgeräte, Kernbohrgeräte und Rotationslaser. Jedes Gerät hat eine Inventarnummer, eine Kategorie und eine übliche Leihdauer. Einzelne Geräte sind ausgemustert oder zur Wartung vorgemerkt. Die Anwendung läuft eigenständig am Lagerplatz-PC, einem Windows-Arbeitsplatz ohne Anbindung an andere Fachsysteme. Der Prüfling hat das Projekt als Auszubildender Fachinformatiker für Anwendungsentwicklung eigenverantwortlich umgesetzt.

1.2 Auftraggeber und Anforderung

Auftraggeber ist die Lagerleitung, vertreten durch irgendein Opfer. Auslöser waren zwei Messgeräte, die im Vorjahr verschwunden sind, ohne dass der letzte Entleiher feststellbar war. Der Auftrag lautet, Ausgabe und Rücknahme digital zu erfassen und überfällige Geräte sichtbar zu machen.

1.3 Ist-Zustand

Jede Ausgabe wird handschriftlich in einer Kladde am Lagerregal festgehalten. Daraus ergeben sich drei Probleme:

1. Ob ein Gerät verfügbar ist, lässt sich nur durch Rückwärtsblättern ermitteln. Der Lagerist schätzt den Aufwand auf dreimal täglich fünf Minuten.
2. Überschrittene Rückgabefristen fallen niemandem auf, weil kein Abgleich der Termine stattfindet.
3. Eine Auswertung zur Auslastung als Grundlage für Nachbeschaffungen ist nicht möglich.

1.4 Soll-Zustand und Projektziel

Nach Projektabschluss verwaltet eine Desktopanwendung den Gerätebestand und erfasst Ausgabe und Rückgabe. Der Status jedes Geräts ist in einer Bestandsübersicht auf einen Blick erkennbar und über Suche und Filter in Sekunden auffindbar. Überfällige Ausleihen werden automatisch ermittelt und lassen sich als CSV-Datei für die Bauleitung exportieren.

1.5 Projektabgrenzung

Nicht Bestandteil des Projekts sind Benutzeranmeldung und Rechteverwaltung, Mehrbenutzerbetrieb, Barcode-Erfassung, Vorabreservierung und Auslastungsstatistiken. Während der Umsetzung wurden zusätzlich die Pflege von Geräte- und Mitarbeiterstammdaten über die Oberfläche zurückgestellt. Die Stammdaten werden einmalig aus CSV-Dateien übernommen. Begründung und Folgen stehen in Abschnitt 8.2.

1.6 Teilaufgaben

1. Datenmodell entwerfen und als SQLite-Datenbank mit EF Core umsetzen.
2. Bestandsdaten aus der bisherigen Kladde, aufbereitet als CSV-Dateien, einmalig importieren.
3. Geschäftsregeln für Ausgabe und Rückgabe in einer Service-Schicht umsetzen.
4. Bestandsübersicht mit Status, Suche und Filter sowie einen Ausleihdialog nach dem MVVM-Muster entwickeln.
5. Überfälligkeitsliste als CSV-Datei exportieren.
6. Anwendung gegen die vereinbarten Testfälle prüfen und dokumentieren.

2 Projektplanung

Das Projekt war laut Projektantrag auf 40 Stunden in vier Phasen angelegt und wurde von einer Person umgesetzt. Die Kosten betragen rund 2.620 Euro, zusätzliche Sachmittel waren nicht nötig.

2.1 Personalplanung

Der Prüfling war für alle Phasen von der Analyse bis zur Dokumentation verantwortlich. Für die Abnahme ist die Lagerleitung als Auftraggeber vorgesehen, für die Einweisung ein Lagerist. Beides hat bis zur Abgabe noch nicht stattgefunden (Abschnitt 6.4 und 7.1).

2.2 Sachmittelplanung

Genutzt wurde der vorhandene Entwicklungsarbeitsplatz mit Windows 11, Visual Studio und dem .NET 10 SDK. Datenbank ist SQLite, die als Datei mit der Anwendung ausgeliefert wird und keinen Datenbankserver benötigt. EF Core und SQLite sind kostenfrei, ebenso das für die Unit-Tests vorgesehene xUnit, für die Entwicklungsumgebung bestehen Lizenzen im Betrieb. Versioniert wurde mit Git auf GitHub.

2.3 Kostenplanung

Die Kosten bestehen fast vollständig aus Personalkosten. Der interne Stundensatz für Entwicklung von 60 Euro deckt Arbeitsplatz, Strom und Räume mit ab. Die Entwicklung ist mit den tatsächlich angefallenen Stunden aus Tab. 2 angesetzt, Einweisung und Abnahme mit ihrem geplanten Aufwand, da beide noch ausstehen.

Posten	Stunden	Satz	Kosten
Entwicklung (Prüfling)	43,0 h	60,00 €/h	2.580,00 €
Einweisung Lagerist (geplant)	0,5 h	35,00 €/h	17,50 €
Abnahme durch Lagerleitung (geplant)	0,5 h	45,00 €/h	22,50 €
Software und Lizenzen			0,00 €
Gesamt			2.620,00 €

Tab. 1: Projektkosten

2.4 Terminplanung

Geplant war laut Projektantrag der Zeitraum vom 11.09.2026 bis zum 29.09.2026 neben dem Tagesgeschäft. Phasen und Soll-Stunden stammen aus dem Projektantrag. Die Abweichungen der Ist-Werte werden in Abschnitt 8.1 begründet.

Nr.	Phase / Arbeitsschritt	Soll	Ist
1	Analysephase	6,0 h	6,0 h
1.1	Projektinitialisierung	2,0 h	1,5 h
1.2	Erstellen des Soll-Konzepts	2,0 h	3,0 h
1.3	Durchführen der Wirtschaftlichkeitsanalyse	2,0 h	1,5 h
2	Entwurfsphase	3,0 h	2,0 h
2.1	Zeitplanung	1,5 h	1,0 h
2.2	Bestimmung der geeigneten Code-Bibliotheken	1,5 h	1,0 h
3	Realisierungsphase	21,0 h	21,0 h
3.1	Gerätestammdaten erfassen	3,0 h	4,5 h
3.2	Mitarbeiterstammdaten erfassen	3,0 h	3,0 h
3.3	Ausleih- und Rückgabefunktion	3,0 h	4,5 h
3.4	Bestandsübersicht	3,0 h	2,5 h
3.5	Überfälligkeitsliste und CSV-Export	5,0 h	6,5 h
3.6	Unit-Tests	4,0 h	0,0 h
4	Projektabschluss und Dokumentation	10,0 h	14,0 h
4.1	Dokumentation des Projekts	5,0 h	12,0 h

Nr.	Phase / Arbeitsschritt	Soll	Ist
4.2	Soll-Ist-Vergleich	1,5 h	1,0 h
4.3	Projektübergabe	2,0 h	0,0 h
4.4	Fazit	1,5 h	1,0 h
	Gesamt	40,0 h	43,0 h

Tab. 2: Terminplanung mit Soll- und Ist-Werten

2.5 Vorgehensmodell

Gewählt wurde ein phasenorientiertes Vorgehen nach dem Wasserfallmodell, weil Umfang und Anforderungen durch den Projektauftrag feststanden. Innerhalb der Implementierung wurde schrittweise gearbeitet: Jede Schicht wurde gebaut, geprüft und committet, bevor die nächste darauf aufsetzte. Erkenntnisse aus späteren Phasen flossen in den Entwurf zurück, etwa der Wechsel von der Leihdauer zum Rückgabedatum (Abschnitt 4.4).

2.6 Projektablauf

Der tatsächliche Ablauf lässt sich aus der Git-Historie nachvollziehen. Die Arbeit verteilte sich auf zwei Abschnitte mit einer Unterbrechung dazwischen.

- **10.09. bis 11.09.2026:** Projektantrag, Projektstruktur, Versionierung und erste Modellklassen.
- **13.09. bis 15.09.2026:** Interfaces, Repositories, Datenbankkontext und die ersten Migrationen.
- **17.09. bis 18.09.2026:** Überarbeitung der Ausleihe, CSV-Import und Bereitstellung der Testdaten im Ausgabeordner.
- **19.09. bis 30.09.2026:** Unterbrechung ohne Projektarbeit.
- **01.10.2026:** Neubewertung des Umfangs. A1 und A2 wurden in der Oberfläche zurückgestellt (Abschnitt 8.2).
- **02.10. bis 03.10.2026:** Service-Schicht mit Geschäftsregeln und Statusermittlung.
- **04.10. bis 06.10.2026:** Oberfläche, Ausleihdialog, Suche, Filter und CSV-Export, abschließend die Abnahmetests T1 und T6.
- **06.10. bis 08.10.2026: Projektdokumentation.**

Der geplante Endtermin am 29.09.2026 wurde damit um gut eine Woche überschritten. Die Gründe stehen in Abschnitt 8.1.

Abb. 1 stellt den geplanten Zeitraum laut Projektantrag dem tatsächlichen Ablauf gegenüber. Deutlich zu sehen sind die Unterbrechung vom 19.09. bis 30.09.2026 und der dadurch verschobene Abschluss.

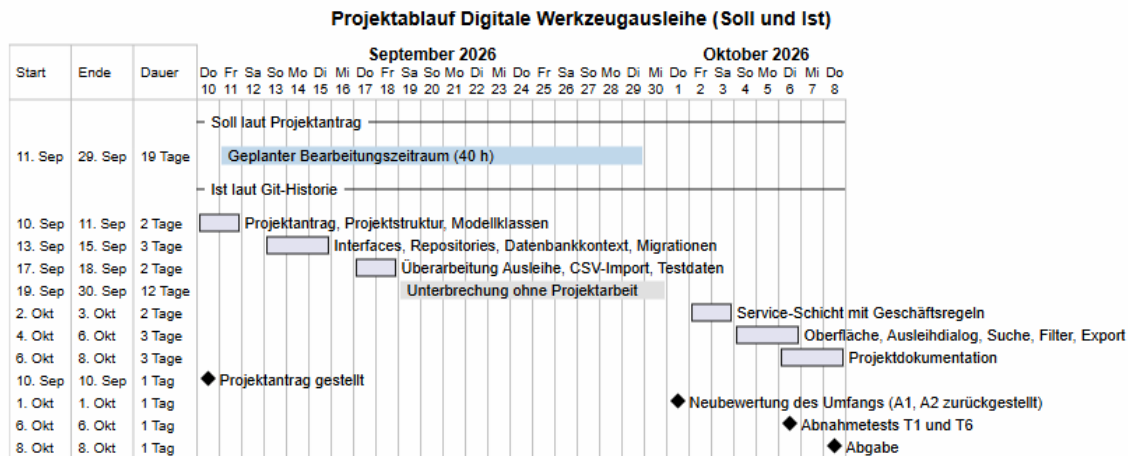


Abb. 1: Projektablauf als Gantt-Diagramm, Soll laut Projektantrag und Ist laut Git-Historie

3 Analysephase

3.1 Make-or-Buy-Entscheidung

Drei Wege wurden verglichen:

Variante	Vorteile	Nachteile
Tabelle in Excel	sofort verfügbar, keine Entwicklung	keine Prüfung auf Doppelausleihe, Status muss von Hand gepflegt werden, Überfälligkeit nicht automatisch
Standardsoftware zur Werkzeugverwaltung	ausgereift, Zusatzfunktionen	laufende Lizenzkosten, Einarbeitung, Funktionsumfang weit über dem Bedarf
Eigenentwicklung	genau auf den Ablauf zugeschnitten, keine Lizenzkosten, erweiterbar	einmaliger Entwicklungsaufwand

Tab. 3: Vergleich der Lösungswege

Entschieden wurde für die Eigenentwicklung. Die Excel-Lösung hätte das Kernproblem, nämlich Fristen und Doppelausleihen, nicht gelöst. Eine Standardsoftware war für 120 Geräte und einen Arbeitsplatz überdimensioniert.

3.2 Wirtschaftlichkeit und Amortisation

Der Lagerist sucht derzeit dreimal täglich rund fünf Minuten in der Kladde. Bei 220 Arbeitstagen sind das 55 Stunden im Jahr. Mit dem Stundensatz des Lageristen von 35 Euro ergibt sich eine Einsparung von 1.925 Euro im Jahr, also rund 160 Euro im Monat.

Amortisationszeit = Projektkosten / Einsparung pro Monat = 2.620,00 € / 160,42 € ≈ 16,3 Monate

Nicht eingerechnet ist der vermiedene Geräteverlust. Im Vorjahr gingen zwei Messgeräte mit zusammen rund 1.400 Euro Wiederbeschaffungswert verloren. Bezieht man ihn ein, verkürzt sich die Amortisation deutlich. Die Rechnung bleibt bewusst vorsichtig, weil sich dieser Nutzen nicht sicher vorhersagen lässt.

3.3 Anwendungsfälle

Hauptakteur ist der Lagerist. Die Bauleitung nutzt die Anwendung nicht selbst, sie erhält die exportierte Überfälligkeitsliste (Abb. 2).

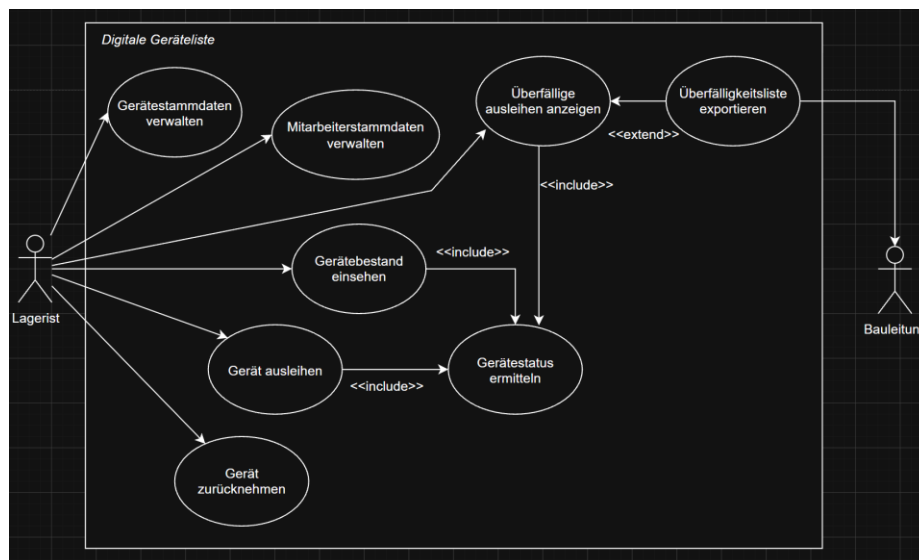


Abb. 2: Anwendungsfalldiagramm aus der Analysephase (10.09.2026)

Das Diagramm zeigt den ursprünglich vereinbarten Umfang. „Gerätestammdaten verwalten“ und „Mitarbeiterstammdaten verwalten“ wurden in der Oberfläche nicht umgesetzt. Die Stammdaten kommen stattdessen aus dem CSV-Import, die Datenschicht für ihre Pflege ist vorbereitet (Abschnitt 5.6). Alle übrigen Anwendungsfälle sind umgesetzt. Übersicht, Ausleihe und Überfälligkeitsliste nutzen dieselbe Statusermittlung im Service.

3.4 Anforderungen und Geschäftsregeln

Die Anforderungen stammen aus dem Projektauftrag. Ihre Umsetzung wird in Abschnitt 8.1 bewertet.

- **A1:** Gerätestammdaten anlegen, bearbeiten und deaktivieren.
- **A2:** Mitarbeiterstammdaten anlegen und bearbeiten.
- **A3:** Ausleihe buchen, das Rückgabedatum ist aus der Standard-Leihdauer vorbelegt.
- **A4:** Rückgabe buchen.

- **A5:** Bestandsübersicht mit Status, Suche und Filter.
- **A6:** Überfälligkeitsliste mit Entleiher und Fristdatum.
- **A7:** Export der Überfälligkeitsliste als CSV-Datei.

Fünf Geschäftsregeln sichern die Daten fachlich ab:

- **R1:** Ein bereits verliehenes Gerät kann nicht erneut ausgeliehen werden.
- **R2:** Ein ausgemustertes Gerät kann nicht ausgeliehen werden.
- **R3:** Die Inventarnummer ist eindeutig.
- **R4:** Das Rückgabedatum liegt nicht vor dem Ausgabedatum.
- **R5:** Der Gerätestatus wird aus den Ausleihen berechnet und nicht gespeichert.

3.5 Qualitätsanforderungen

Die Anwendung soll ohne Schulung bedienbar sein, die Übersicht muss bei 120 Geräten ohne spürbare Wartezeit laden. Fehlbedienungen dürfen keine inkonsistenten Daten erzeugen. Die Geschäftsregeln müssen ohne Datenbank und Oberfläche testbar sein.

4 Entwurfsphase

4.1 Zielplattform

Die Anwendung ist eine WPF-Desktopanwendung auf .NET 10. WPF wurde gewählt, weil der Lagerplatz-PC unter Windows läuft und keine Server-Infrastruktur vorhanden ist. Als Datenbank dient SQLite: Sie liegt als einzelne Datei neben der Anwendung und erfordert keine Installation. Der Zugriff erfolgt über EF Core, das Tabellen aus den Modellklassen erzeugt und Schemaänderungen über Migrationen nachvollziehbar macht.

Eingesetzte Technologien im Überblick:

- **Programmiersprache:** C# mit aktivierten Nullable Reference Types
- **Framework:** .NET 10 (Zielplattform net10.0-windows, SDK 10.0.401)
- **Oberfläche:** WPF mit XAML, MVVM ohne Zusatzbibliothek, mit eigener ViewModelBase und eigenem RelayCommand
- **Datenbank:** SQLite als einzelne Datei im Programmordner
- **Datenzugriff:** Entity Framework Core 10.0.12 mit den NuGet-Paketen Microsoft.EntityFrameworkCore.Sqlite und Microsoft.EntityFrameworkCore.Design. Migrationen mit dem Kommandozeilenwerkzeug dotnet-ef 10.0.12
- **Datenaustausch:** CSV-Dateien mit Semikolon als Trennzeichen in UTF-8, für den Import der Bestandsdaten und den Export der Überfälligkeitsliste
- **Entwicklungsumgebung:** Visual Studio 2026 (Version 18.9 bis 18.10) unter Windows 11
- **Versionsverwaltung:** Git 2.54, Repository auf GitHub
- **Diagramme:** PlantUML für das Gantt-Diagramm, draw.io (app.diagrams.net) für das Anwendungsfalldiagramm
- **Geplant:** xUnit für die Unit-Tests (Abschnitt 6.2)

4.2 Architektur

Abb. 3 zeigt die Schichten und ihre Abhängigkeiten. Die Regel lautet: Jede Schicht kennt nur die Schicht unter sich, und die Fachlogik kennt die Datenbank nur über Schnittstellen. Eine Ausnahme beim CSV-Export ist in Abschnitt 8.2 beschrieben.

Nur die Datenschicht kennt die Datenbank

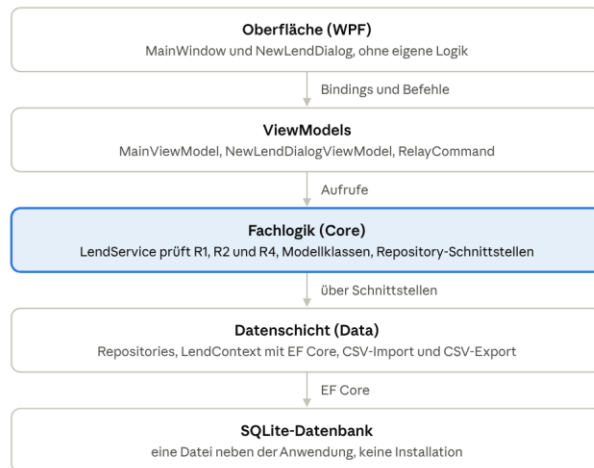


Abb. 3: *Schichtenarchitektur*. Hervorgehoben ist die Fachlogik: Sie kennt die Datenbank nur über Schnittstellen und ist dadurch ohne Datenbank testbar.

Die Repositories setzen das Repository-Muster um. Der Service bekommt sie über ihre Schnittstellen im Konstruktor übergeben. Für Unit-Tests lassen sich so einfache Test-Implementierungen einsetzen, die mit Listen statt mit einer Datenbank arbeiten. Zusammengesetzt werden alle Objekte an genau einer Stelle, beim Programmstart in der Klasse App.

4.3 Datenmodell

Abb. 4 zeigt das Datenmodell mit den Tabellen und Datentypen, die EF Core in SQLite anlegt. Alle Beziehungen sind 1:n. Jedes Gerät gehört zu genau einer Kategorie, und jede Ausleihe betrifft genau ein Gerät. Zum Mitarbeiter bestehen zwei Beziehungen, als Entleiher (BorrowedById) und als ausgebender Lagerist (LendById). Die Ausleihe löst damit die n:m-Beziehung zwischen Geräten und Mitarbeitern auf. Da SQLite keine eigenen Typen für Datum und Wahrheitswert kennt, speichert EF Core sie als TEXT und INTEGER.

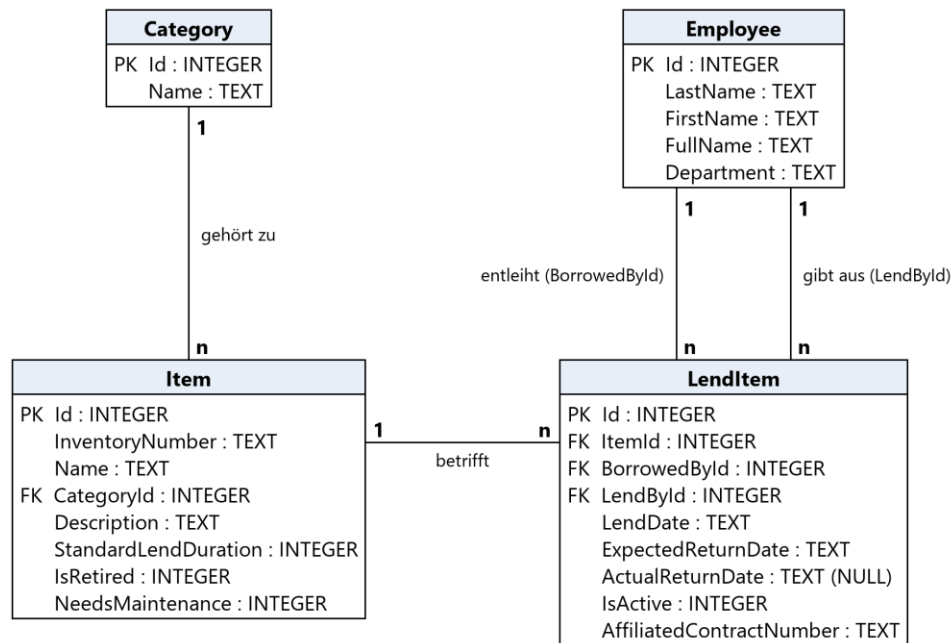


Abb. 4: Entity-Relationship-Modell der Datenbank (PK = Primärschlüssel, FK = Fremdschlüssel)

Zwei Entscheidungen prägen das Modell. Erstens hat das Gerät kein Statusfeld (R5). Der Status ergibt sich aus der offenen Ausleihe, ein gespeichertes Feld könnte der Historie widersprechen. Dass je Gerät höchstens eine Ausleihe offen ist (R1), lässt sich nicht über einen Fremdschlüssel abbilden und wird deshalb im Service geprüft. Zweitens ist das Löschverhalten aller Fremdschlüssel auf Restrict gesetzt, damit beim Löschen von Stammdaten keine Ausleihhistorie verloren geht.

4.4 Entwurfsentscheidungen

- **Rückgabedatum statt Leihdauer:** Der Lagerist wählt im Dialog den vereinbarten Rückgabetag (DateTime?). Eine Leihdauer in Tagen hätte zwischen Oberfläche und Fachlogik hin und her umgerechnet werden müssen. Ohne Datum gilt die Standard-Leihdauer des Geräts, und R4 wird zum direkten Datumsvergleich.
- **Ergebnisobjekt statt Wahrheitswert:** TransactionResult liefert neben dem Erfolg auch die Meldung für den Lageristen. Es entsteht nur über Success oder Failure, sodass kein widersprüchliches Ergebnis möglich ist (Abb. 17).
- **Stichtag als Parameter:** Die Statusermittlung erhält das heutige Datum als Parameter, statt es selbst zu lesen. So lässt sie sich mit festen Daten testen.
- **Datumswerte ohne Uhrzeit:** Fristen sind Kalendertage, der Service vergleicht nur den Datumsanteil.
- **Ausgeber nur aus dem Lager:** Der Dialog bietet als Ausgeber nur Mitarbeiter der Abteilung Lager an. Die Abteilung ist derzeit als Text hinterlegt, eine eigene Tabelle wäre sauberer (Abschnitt 8.5).
- **Ein gemeinsamer Datenbankkontext:** Alle Repositories teilen sich eine LendContext-Instanz, die die Klasse App beim Programmstart erzeugt. Mit getrennten Kontexten wüssten sie nichts von den Änderungen der anderen.

4.5 Benutzeroberfläche

Das Hauptfenster zeigt links die Bestandsübersicht als Tabelle mit Inventarnummer, Bezeichnung, Kategorie, Ausgeber („Entleihen von“), Entleiher („Entleihen an“), Status, Ausgabedatum, Frist und Vertragsnummer. Rechts stehen die Aktionen: Filter für offene und überfällige Verleihe, Export, Neuer Verleih und Rückgabe. Buttons sind nur aktiv, wenn die Aktion für die markierte Zeile zulässig ist. So sieht der Lagerist R1 und R2 schon an der Oberfläche. Die umgesetzte Oberfläche zeigen die Screenshots in Anhang A5.

4.6 Maßnahmen zur Qualitätssicherung

Die Geschäftsregeln werden doppelt gesichert: in der Oberfläche über gesperrte Buttons und im Service über Prüfungen vor jeder Buchung. Da der Service die Datenbank nur über Interfaces kennt, ist er für Unit-Tests vorbereitet (Abschnitt 6.2). Jede Schemaänderung erhält eine eigene Migration, jeder Arbeitsschritt einen eigenen Git-Commit.

5 Implementierungsphase

Umgesetzt wurde von unten nach oben: zuerst Datenbank und Import, dann die Geschäftslogik, zuletzt die Oberfläche. Jede Schicht wurde geprüft, bevor die nächste darauf aufbaute.

5.1 Datenhaltung

Der Datenbankkontext `LendContext` beschreibt die vier Tabellen. Den Pfad zur Datenbankdatei bildet er aus dem Programmverzeichnis, damit die Anwendung auf jedem Rechner lauffähig ist. Das Löschverhalten wird in `OnModelCreating` je Beziehung festgelegt, hier am Beispiel des Entleihers:

```
modelBuilder.Entity<LendItem>()  
    .HasOne(li => li.BorrowedBy)  
    .WithMany()  
    .HasForeignKey(li => li.BorrowedById)  
    .onDelete>DeleteBehavior.Restrict);
```

Das Schema entstand über drei Migrationen: `InitialCreate` legt die Tabellen an, `RestrictDeleteBehaviour` schaltet die Löschweitergabe ab, und `RemoveEmployeeEmail` entfernt ein Feld, das keine Anforderung benötigt.

5.2 Datenimport

Die Bestandsdaten aus der Kladde wurden als vier CSV-Dateien aufbereitet: 8 Kategorien, 120 Geräte, 45 Mitarbeiter und 53 Ausleihen. Sie werden mit der Anwendung ausgeliefert und beim ersten Start importiert, sofern die Datenbank leer ist. Die Ids werden aus den Dateien übernommen, damit die Verweise der Ausleihen auf Geräte und Mitarbeiter gültig bleiben. Startet die Anwendung mit bereits gefüllter Datenbank, wird der übersprungene Import in eine Logdatei je Tag geschrieben, ebenso jeder Fehler beim Import.

5.3 Geschäftslogik

Die Klasse `LendService` setzt die Geschäftsregeln um. Vor jeder Ausleihe prüft sie die Regeln nacheinander und bricht beim ersten Verstoß mit einer Meldung ab. Abb. 5 zeigt den Ablauf. Den vollständigen Code von Ausleihe und Rückgabe zeigen Abb. 15 und Abb. 16 in Anhang A6.

Erst nach drei bestandenen Prüfungen wird die Ausleihe gespeichert

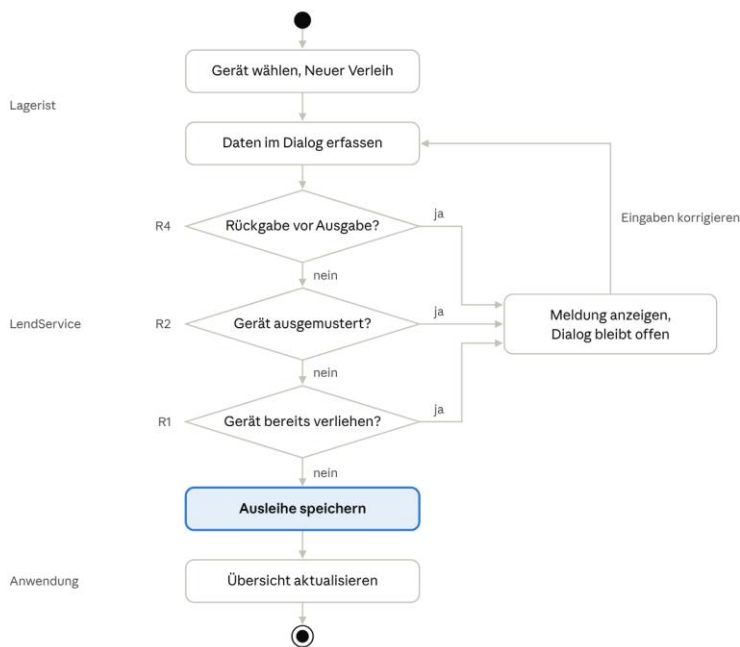


Abb. 5: Ablauf einer Ausleihe. Jede verletzte Regel führt zurück in den Dialog, sodass der Lagerist seine Eingabe korrigieren kann, ohne neu zu beginnen.

Die Bestandsübersicht ermittelt den Status jedes Geräts. Damit nicht für jedes der 120 Geräte eine eigene Datenbankabfrage nötig ist, lädt der Service alle offenen Ausleihen einmal und legt sie nach Geräte-Id in einem Dictionary ab. Das Gruppieren schützt dabei vor einem Absturz, falls Altdaten doch zwei offene Ausleihen für ein Gerät enthalten:

```
Dictionary<int, LendItem> openLends = _lendItemRepository.GetAllLendItems()
    .Where(l => l.ActualReturnDate == null)
    .GroupBy(l => l.ItemId)
    .ToDictionary(g => g.Key, g => g.First());
```

5.4 Benutzeroberfläche

Die Oberfläche folgt dem MVVM-Muster. Die Fenster binden nur an Eigenschaften und Befehle ihrer ViewModels. ViewModelBase meldet geänderte Werte an die Oberfläche. RelayCommand verbindet Buttons mit Methoden und sperrt sie, solange die Aktion für die markierte Zeile nicht zulässig ist:

```
ReturnCommand = new RelayCommand(_ => ReturnSelected(), _ => SelectedRow?.LendItemID != null);
LendCommand = new RelayCommand(_ => OpenLendDialog(), _ => SelectedRow?.Status == ItemStatus.Available);
```

Der Ausleihdialog ruft den Service selbst auf und bleibt bei einer abgelehnten Buchung offen, damit der Lagerist korrigieren kann. Bei Erfolg schließt er sich über ein Ereignis des ViewModels.

Die Suche vergleicht den Suchtext mit Inventarnummer, Bezeichnung, Kategorie, Status, beiden Namen und den Datumsangaben. Die Filterbuttons setzen einen Status, wobei „Offene Verleihe“ auch

überfällige Geräte zeigt. Ist ein Suchtext eingegeben, hat er derzeit Vorrang vor dem Statusfilter. Die Methode und ihre Schwächen zeigt Abb. 18 in Anhang A6.

5.5 Export

Der Export schreibt die überfälligen Geräte der aktuellen Übersicht in eine CSV-Datei, deren Speicherort der Lagerist wählt. Sie enthält die Spalten der Übersicht außer dem Status und zusätzlich die Zahl der Tage über der Frist. Trennzeichen ist das Semikolon, Semikolons in Texten werden durch Kommas ersetzt. Die Datei wird in UTF-8 mit Kennung geschrieben, damit Excel Umlaute korrekt anzeigt. Gibt es keine überfälligen Ausleihen, erscheint ein Hinweis statt einer leeren Datei.

```
int daysOverdue = (dateToCheck.Date - row.ExpectedReturnDate!.Value.Date).Days;  
// ...  
File.WriteAllLines(filePath, lines, Encoding.UTF8);  
  
private static string Clean(string? value) => (value ?? string.Empty).Replace(";",  
",");
```

Die Anwendung ist für deutsche Systemeinstellungen ausgelegt. Die Screenshots entstanden jedoch auf dem privaten Rechner des Prüflings mit englischem Windows und Excel. Deshalb zeigt der Ausleihdialog das Datumsformat Monat/Tag/Jahr (Abb. 10), und Speichern-Dialog sowie Rückfragen erscheinen auf Englisch (Abb. 13). Excel trennt die CSV-Spalten nicht, weil die englische Version ein Komma erwartet (Abb. 14). Übersicht, Suche und Export verwenden dagegen fest das Format TT.MM.JJJJ.

5.6 Vorbereitete Funktionen ohne Oberfläche

Folgende Funktionen sind in der Datenschicht umgesetzt, aber noch nicht an die Oberfläche angebunden. Sie entstanden vor der Neubewertung des Umfangs am 01.10.2026 und sind die Grundlage der Nacharbeiten in Abschnitt 8.5.

- **Gerätstammdaten (A1):** Das ItemRepository legt Geräte an und ändert sie, einschließlich der Kennzeichen für Ausmusterung und Wartung. Geräte werden nur ausgemustert (IsRetired), nie gelöscht, damit die Ausleihhistorie vollständig bleibt. Für R3 prüft CheckInventoryNumberDuplicate, ob ein anderes Gerät die Inventarnummer schon nutzt. Anlegen und Ändern rufen diese Prüfung noch nicht auf, das soll später der Service übernehmen.
- **Mitarbeiterstammdaten (A2):** Das EmployeeRepository legt Mitarbeiter an und ändert sie. Beim Ändern wird der vollständige Name neu gebildet, und die Methode meldet zurück, ob etwas gespeichert wurde.
- **Korrektur einer Ausleihe:** ChangeLendItem ändert eine offene Ausleihe und schreibt alte und neue Werte in die Logdatei des Tages. So bleibt nachvollziehbar, wer ein Gerät ursprünglich hatte. Zurückgegebene Ausleihen lassen sich nicht mehr ändern.
- **Wartungskennzeichen:** NeedsMaintenance wird beim Import übernommen (derzeit fünf Geräte), von Statusermittlung und Ausleihe aber noch nicht berücksichtigt. Ein Gerät in Wartung kann deshalb momentan ausgeliehen werden.
- **GetAllOverdueLendItems:** Diese ältere Abfrage im LendItemRepository richtet sich nach dem Systemdatum und wird nicht mehr aufgerufen. Übersicht, Filter und Export nutzen die Statusermittlung des Service mit Stichtag (Abschnitt 4.4). Die Methode kann entfallen.

6 Qualitätssicherung und Abnahme

Für die Abnahme sind acht Tests an der laufenden Anwendung festgelegt (Tab. 4). Unit-Tests sind vorbereitet und werden nachgeholt.

6.1 Testkonzept

Die Abnahmetests stammen aus dem Projektauftrag und liefern mit den importierten Bestandsdaten. Die geplanten Unit-Tests prüfen den LendService isoliert, mit Test-Implementierungen der Repositories statt einer Datenbank. Die Überfälligkeit wird dabei mit festem Stichtag geprüft, weil sich die Zahl überfälliger Ausleihen in den Testdaten täglich ändert: am 08.10.2026 sind es 6, am 14.10.2026 bereits 18.

6.2 Geplante Unit-Tests

Als erste Tests sind vorgesehen:

- **IsOverdueAt_OpenLendAfterDeadline_ReturnsTrue:** Eine offene Ausleihe gilt nach Ablauf der Frist als überfällig.
- **IsOverdueAt_ReturnedLend_ReturnsFalse:** Eine zurückgegebene Ausleihe gilt nie als überfällig, auch wenn die Frist abgelaufen ist.
- **BorrowItem_ItemAlreadyLent_Fails:** Ein bereits verliehenes Gerät kann nicht erneut ausgeliehen werden (R1), und es entsteht keine neue Ausleihe.

Dafür sind nur ein Testprojekt und je eine Test-Implementierung der drei Repository-Interfaces nötig, die Anwendung selbst muss nicht geändert werden.

6.3 Abnahmetests

Nr.	Testfall	Erwartet	Ergebnis
T1	Ausleihe eines verfügbaren Geräts	Ausleihe gespeichert, Status wechselt auf verliehen	bestanden: Gerät 1003 am 06.10.2026 an Tobias Schwarz, Frist 13.10.2026 aus der Standard-Leihdauer. Erneut bestanden am 08.10.2026: Gerät 1001 an Stefan Becker, ausgegeben von Jonas Wagner, Rückgabe bis 30.10.2026, Status danach LentOut (Abb. 10 und Abb. 11)
T2	Ausleihe eines verliehenen Geräts (R1)	Ausleihe nicht möglich	Button gesperrt, Service lehnt ab
T3	Ausleihe eines ausgemusterten Geräts (R2)	Ausleihe nicht möglich	Inventarnummer 1018: Button gesperrt

Nr.	Testfall	Erwartet	Ergebnis
T4	Doppelte Inventarnummer (R3)	Ablehnung	entfällt in der Oberfläche, da A1 zurückgestellt; Prüfmethode im Repository vorhanden
T5	Rückgabedatum vor Ausgabedatum (R4)	Meldung, Dialog bleibt offen	bestanden, Meldung: Invalid return date, Dialog bleibt offen
T6	Rückgabe einer offenen Ausleihe	Rückgabedatum gesetzt, Gerät wieder verfügbar	bestanden: Ausleihe 31, Gerät 1004, am 06.10.2026 zurückgegeben
T7	Überfällige Ausleihen anzeigen	nur Geräte mit abgelaufener Frist	bestanden: am 08.10.2026 sechs Geräte (1004, 1015, 1033, 1037, 1084, 1098), siehe Abb. 12
T8	Export der Überfälligkeitsliste	CSV-Datei mit Kopfzeile, eine Zeile je überfälligem Gerät	bestanden: Datei mit Kopfzeile und sechs Zeilen, Umlaute korrekt, siehe Abb. 14

Tab. 4: Abnahmetests

T1 und T6 liefen am 06.10.2026 auf dem damaligen Datenstand. Für die Screenshots wurde die Datenbank am 08.10.2026 neu aus den CSV-Dateien aufgebaut, deren Datumswerte zuvor auf den Abgabezeitraum verschoben wurden. Deshalb ist Gerät 1004 in Abb. 9 bis 12 wieder überfällig.

Zu T2 und T3: Die Oberfläche verhindert beide Fälle bereits, indem sie den Button sperrt. Der Service prüft vor dem Speichern trotzdem erneut. Damit bleiben die Regeln auch dann gesichert, wenn später eine weitere Oberfläche denselben Service nutzt.

6.4 Abnahme durch den Auftraggeber

Eine Abnahme durch die Lagerleitung hat bis zur Abgabe nicht stattgefunden. Sie ist für die Zeit nach der Abgabe vorgesehen. Grundlage sind dann die Abnahmetests aus Tab. 4, die bisher nur der Prüfling selbst durchgeführt hat.

7 Einführung und Dokumentation

7.1 Einführung

Die Anwendung wird als Ordner mit Laufzeitbibliotheken und den vier CSV-Dateien auf den Lagerplatz-PC kopiert und ist ohne Installation lauffähig. Vor dem Produktivstart wird die Datenbank über die Migrationen angelegt, beim ersten Start übernimmt die Anwendung die Bestandsdaten. Eine

halbstündige Einweisung der Lageristen ist geplant, hat bis zur Abgabe aber noch nicht stattgefunden. Die Kladde wird bis 31.12.2026 parallel weitergeführt und danach archiviert.

7.2 Dokumentation

- **Anwenderdokumentation:** eine Kurzanleitung für den Lageristen mit Screenshots zu Übersicht, Ausleihe, Filtern und Export (Anhang A5).
- **Entwicklerdokumentation:** die Git-Historie mit einem Commit je Arbeitsschritt, die Kommentare im Quellcode und die Kapitel 4 und 5 dieser Dokumentation.
- **Projektdokumentation:** das vorliegende Dokument.

8 Projektabschluss

8.1 Soll-Ist-Vergleich

Nr.	Anforderung	Ergebnis
A1	Gerätestammdaten pflegen	zurückgestellt, Daten aus CSV-Import
A2	Mitarbeiterstammdaten pflegen	zurückgestellt, Daten aus CSV-Import
A3	Ausleihe buchen	erfüllt
A4	Rückgabe buchen	erfüllt
A5	Bestandsübersicht mit Suche und Filter	erfüllt
A6	Überfälligkeitsliste	erfüllt, als Filter der Übersicht
A7	CSV-Export	erfüllt

Tab. 5: Umsetzung der Anforderungen

Der Gesamtaufwand lag mit 43 Stunden um 3 Stunden über Plan (Tab. 2). Analyse und Entwurf blieben mit 8 statt 9 Stunden knapp darunter. Die Realisierung traf mit 21 Stunden genau den Plan, allerdings nur, weil die 4 Stunden für Unit-Tests entfielen. Mehr Zeit als geplant brauchten die Stammdatenerfassung mit 7,5 statt 6 Stunden, obwohl sie nur in der Datenschicht umgesetzt ist, die Ausleih- und Rückgabefunktion mit 4,5 statt 3 Stunden sowie Überfälligkeitsliste und Export mit 6,5 statt 5 Stunden. Ursachen waren vor allem die Einarbeitung in EF Core und MVVM, eine fehlerhafte erste Migration und mehrere Überarbeitungen der Service-Schicht. Der Projektabschluss brauchte 14 statt 10 Stunden. Die Dokumentation kostete 12 statt 5 Stunden, weil der Aufwand für Text, Abbildungen und Testnachweise zu knapp angesetzt war. Die Projektübergabe entfiel, weil Abnahme und Einweisung noch ausstehen (Abschnitt 6.4 und 7.1).

Der Endtermin 29.09.2026 wurde nicht gehalten: Die Anwendung war am 06.10.2026 fertig, die Dokumentation am 08.10.2026. Hauptursache war die Unterbrechung vom 19.09. bis 30.09.2026

(Abschnitt 2.6). Bei einem echten Kundenauftrag wäre die Zurückstellung von A1 und A2 früher mit dem Auftraggeber abzustimmen gewesen.

8.2 Abweichungen vom Entwurf

- **A1 und A2 nur über Import:** Am 01.10.2026 wegen des Zeitbudgets zurückgestellt. Für die Ablösung der Kladde war die Erfassung von Ausleihen vorrangig.
- **Änderungen am Datenmodell und Entwurf:** Statusfeld am Gerät entfernt (R5), Rückgabedatum statt Leihdauer, Löschweitergabe abgeschaltet, Ausgeber auf das Lager beschränkt und E-Mail-Adresse der Mitarbeiter entfernt. Die Gründe stehen in Abschnitt 4.3, 4.4 und 5.1.
- **Protokollierung ergänzt:** Änderungen an Ausleihen werden mit alten und neuen Werten protokolliert, weil fehlende Nachvollziehbarkeit das Ausgangsproblem war.
- **Filter „Offene Verleihe“:** Er schließt überfällige Geräte ein, weil sie ebenfalls verliehen und die dringendsten sind.
- **Unit-Tests verschoben:** Die Zeit floss in Einarbeitung und Oberfläche, die Tests werden nachgeholt (Abschnitt 6.2).
- **Suche und Status:** Suchtext und Statusfilter lassen sich noch nicht kombinieren, und der Status erscheint mit den englischen Enum-Namen (Abb. 18).
- **Export direkt aus dem ViewModel:** Das MainViewModel ruft CSVExport aus der Datenschicht direkt auf und umgeht damit die Schichtenregel aus Abschnitt 4.2. Auslöser war der Zeitdruck kurz vor Projektende.
- **ViewModels kennen die Oberfläche:** Das MainViewModel öffnet den Ausleihdialog selbst und nutzt SaveFileDialog, beide ViewModels zeigen Meldungen über MessageBox. Nach MVVM sollten ViewModels keine Fenster kennen, damit sie ohne Oberfläche testbar sind. In kleinen Anwendungen ist das verbreitet, widerspricht aber der Trennung aus Abschnitt 4.2.

Die letzten vier Punkte sind als Nacharbeit vorgesehen (Abschnitt 8.5).

8.3 Lessons Learned

- **Migrationen sind Quellcode.** Das Löschen der Datenbankdatei setzt die Migrationen nicht zurück. Eine einzeln gelöschte Migration bei gehaltenem Snapshot erzeugte eine leere Folgemigration.
- **Verknüpfte Daten müssen ausdrücklich geladen werden.** Fehler zeigten sich erst beim zweiten Programmstart, weil der Import beim ersten Start alle Objekte im Speicher verknüpft hatte.
- **Binding-Fehler sind still.** Ein falsch geschriebener Eigenschaftsname lässt einen Button wirkungslos, ohne Compilerfehler. Das Ausgabefenster von Visual Studio zeigt diese Fehler beim Start.
- **Früh und oft ausführen.** Mehrere Funktionen waren tagelang nur kompiliert. Erst der Lauf mit echten Daten zeigte, ob sie funktionieren.
- **Schichtentrennung zahlt sich aus.** Weil der Service nur Schnittstellen kennt, ließ sich die Oberfläche zuletzt aufsetzen, und die Unit-Tests können ohne Umbau ergänzt werden.

8.4 Fazit

Das Kernziel ist erreicht. Fünf von sieben Anforderungen sind umgesetzt, die Regeln R1, R2, R4 und R5 greifen, R3 ist in der Datenschicht vorbereitet. Ausgabe und Rückgabe werden digital gebucht, und zu

jedem Gerät ist nachvollziehbar, wer es zuletzt hatte. Damit ist das Problem gelöst, das den Auftrag ausgelöst hat.

Nicht erreicht wurden der geplante Termin und der volle Umfang, Stammdatenpflege und Unit-Tests fehlen noch. Rückblickend waren Dokumentation und Einarbeitung in EF Core und MVVM zu knapp geplant. Die Einarbeitung hatte keinen eigenen Posten und ging zulasten der Unit-Tests. Bei einem nächsten Projekt würde der Prüfling dafür Zeit einplanen und den Umfang früher mit dem Auftraggeber abstimmen.

Fachlich hat das Projekt vor allem gezeigt, wie viel eine saubere Schichtentrennung wert ist. Die zurückgestellten Funktionen lassen sich ergänzen, ohne die bestehenden Teile umzubauen.

8.5 Ausblick

Als Nächstes stehen die Nacharbeiten aus diesem Projekt an:

- Unit-Tests für den LendService (Abschnitt 6.2).
- Oberflächen für die Stammdatenpflege (A1, A2) auf Basis der Repository-Methoden aus Abschnitt 5.6, mit Aufruf der Dublettenprüfung (R3) im Service.
- Ein Status „In Wartung“, der Geräte mit Wartungskennzeichen für die Ausleihe sperrt.
- Refresh überarbeiten, damit Suche und Statusfilter zusammenwirken, und den Status auf Deutsch anzeigen (Abb. 18).
- Den CSV-Export hinter ein Interface in der Fachlogik legen und Dialoge über einen Dialog-Service öffnen, damit die ViewModels weder Datenschicht noch Fenster kennen (Abschnitt 8.2).
- Den Export zusätzlich als Excel-Datei anbieten, unabhängig von den Ländereinstellungen.
- Eine Oberfläche für die Korrektur von Ausleihen mit dem vorhandenen Änderungsprotokoll.

Darüber hinaus bieten sich als Folgeaufträge eine Auslastungsstatistik als Grundlage für Nachbeschaffungen und eine Barcode-Erfassung bei der Ausgabe an. Abteilungen sollten dann eine eigene Tabelle erhalten, statt als Text gespeichert zu werden. Für mehrere Arbeitsplätze müsste SQLite durch einen Datenbankserver ersetzt werden. Durch die Repository-Schicht beträfe das nur die Datenschicht.

Anhang

Der Anhang enthält Unterlagen, die den Textteil belegen, aber dessen Umfang sprengen würden.

A1 Projektantrag



Projektantrag_Alexa
nder_Lang.pdf

A2 Zeitplanung mit Ist-Werten

Die Ist-Werte sind direkt in Tab. 2 in Abschnitt 2.4 eingetragen.

A3 Klassendiagramm

Das Klassendiagramm ist zur besseren Lesbarkeit in drei Teile aufgeteilt: Fachlogik mit Datenmodell (Abb. 6), Datenschicht (Abb. 7) und Oberflächenlogik (Abb. 8). Klassen aus einem anderen Teil erscheinen nur mit ihrem Namen. Statische Member sind unterstrichen, Events sind mit «event» gekennzeichnet. Klassen, die ein Interface implementieren, zeigen nur ihre Konstruktoren und zusätzlichen Member, die Methoden des Interface stehen beim Interface selbst. Bei Parametern sind die Datentypen weggelassen, lange Parameterlisten sind mit (...) abgekürzt.

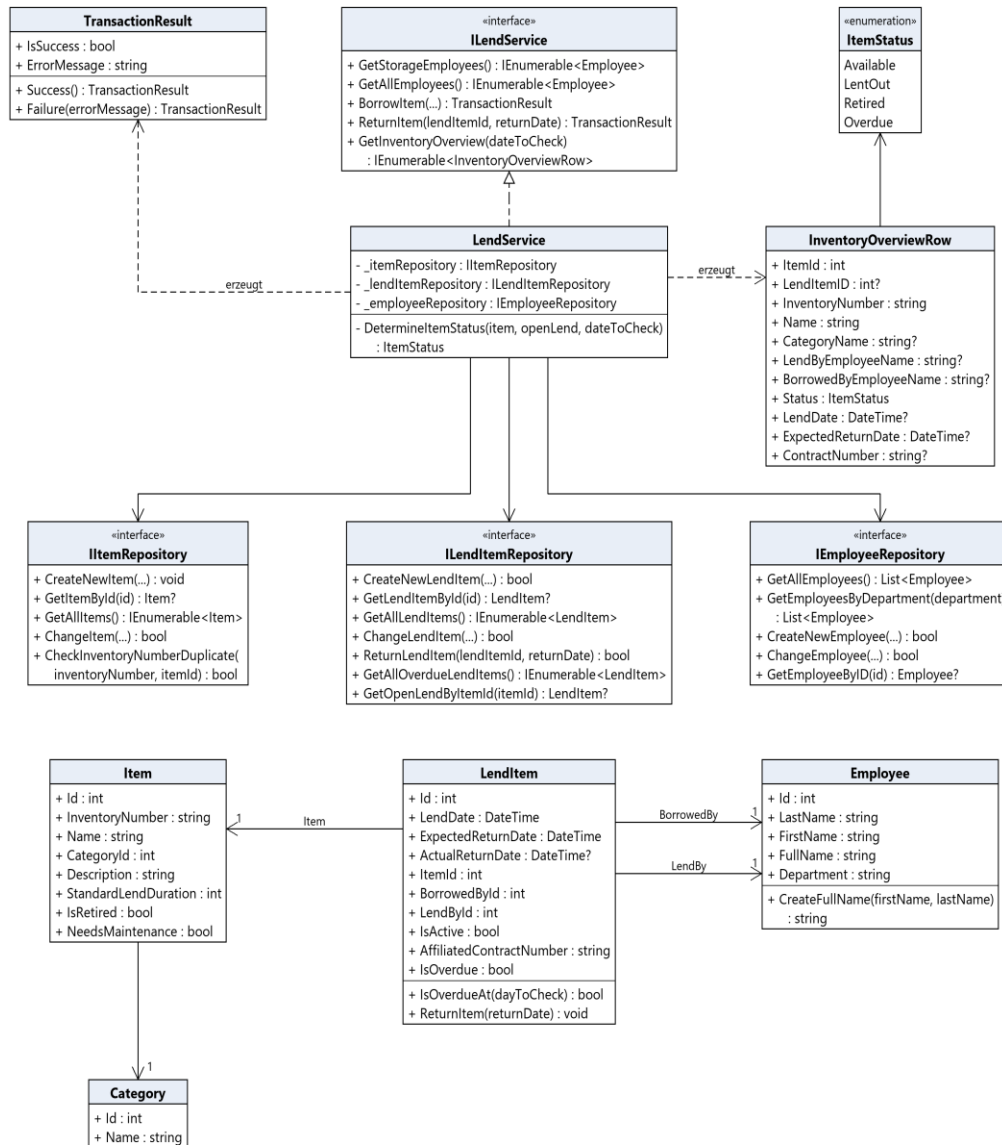


Abb. 6: Klassendiagramm Teil 1, Fachlogik und Datenmodell

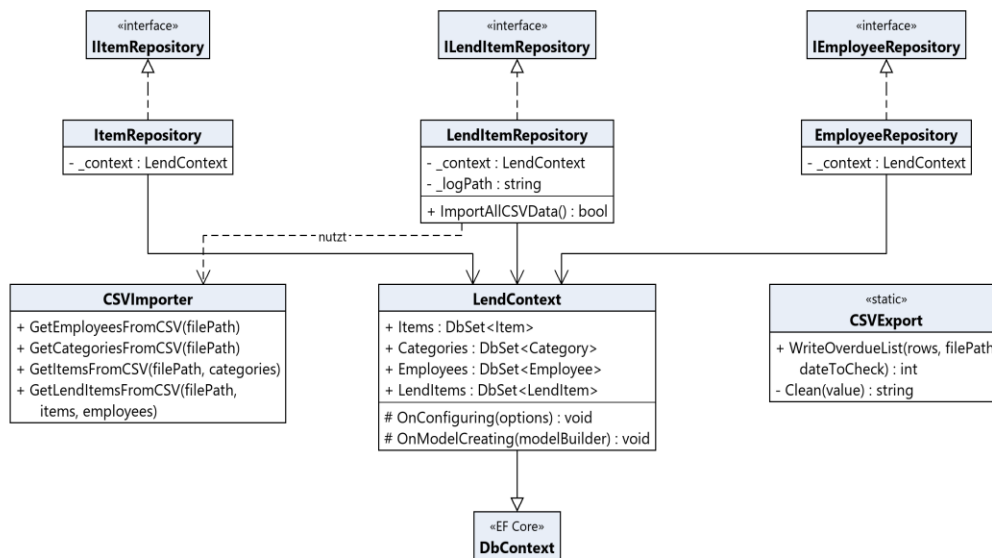


Abb. 7: Klassendiagramm Teil 2, Datenschicht

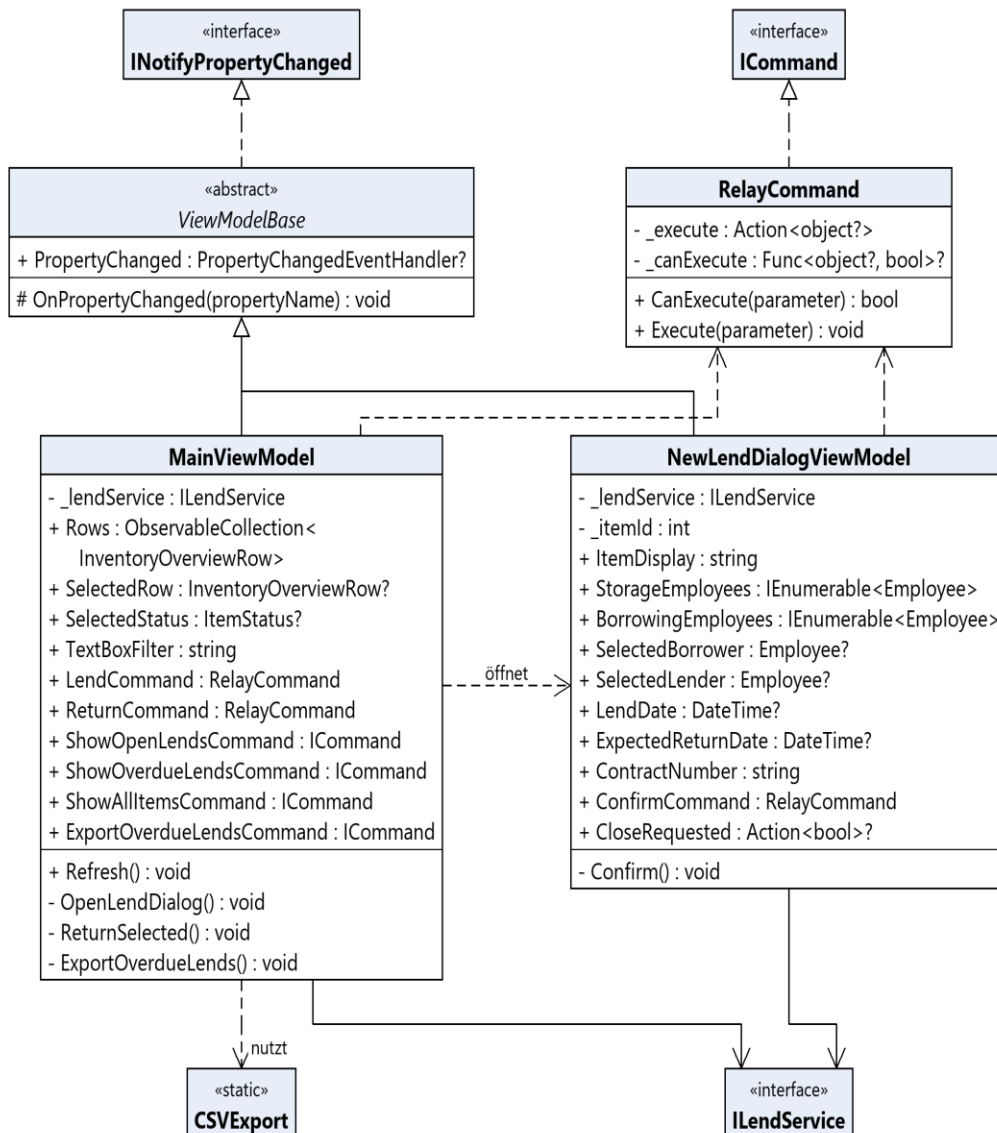


Abb. 8: Klassendiagramm Teil 3, Oberflächenlogik

A4 Screenshots der Anwendung

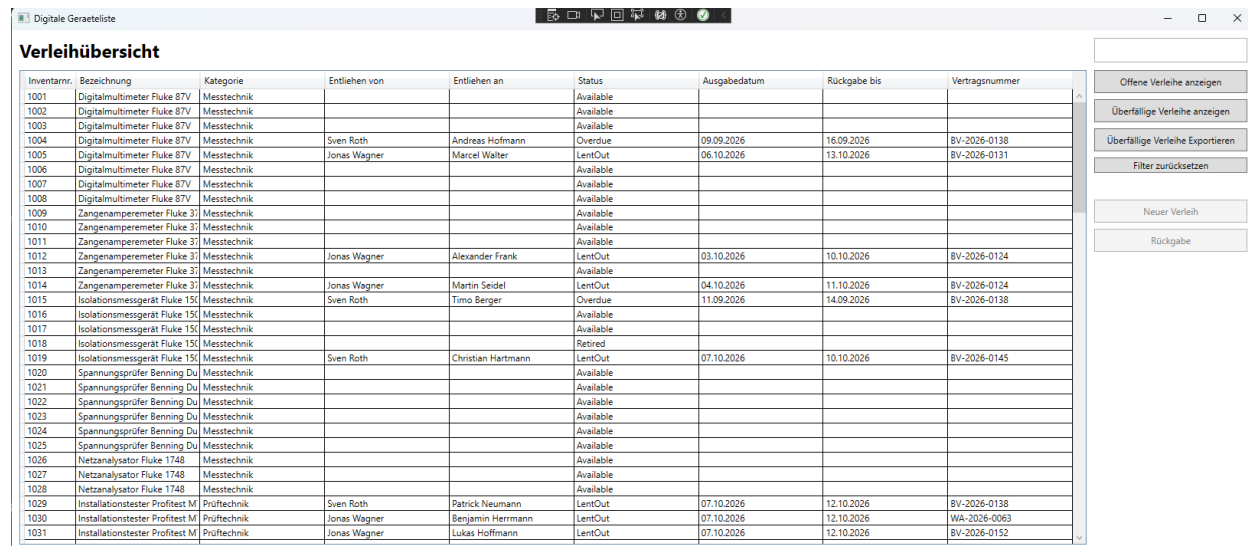
Die Screenshots sind in die Anwenderdokumentation in Anhang A5 eingebunden (Abb. 9 bis Abb. 14).

A5 Anwenderdokumentation

Diese Kurzanleitung richtet sich an die Lageristen. Sie beschreibt alle Aufgaben, die mit der Anwendung erledigt werden. Die Screenshots stammen von einem Rechner mit englischen Spracheinstellungen. Mit deutschem Windows erscheinen Datumfelder im Format TT.MM.JJJJ und Systemdialoge auf Deutsch (Abschnitt 5.4).

Programmstart

Die Anwendung wird über die Datei Digitale_Geraeteliste.exe im Programmordner gestartet. Beim ersten Start übernimmt sie die Bestandsdaten automatisch. Danach öffnet sich das Hauptfenster mit allen Geräten.



The screenshot shows the 'Digitale Geraeteliste' application window. The main area contains a table with columns: Inventarnr., Bezeichnung, Kategorie, Entliehen von, Entliehen an, Status, Ausgabedatum, Rückgabe bis, and Vertragsnummer. The table lists various equipment items, including digital multimeters, Zangenampere meter, Isolationsschere, and Netzanalysator. The sidebar on the right contains buttons for 'Offene Verleihe anzeigen', 'Überfällige Verleihe anzeigen', 'Überfällige Verleihe Exportieren', 'Filter zurücksetzen', 'Neuer Verleih', and 'Rückgabe'.

Inventarnr.	Bezeichnung	Kategorie	Entliehen von	Entliehen an	Status	Ausgabedatum	Rückgabe bis	Vertragsnummer
1001	Digitalmultimeter Fluke 87V	Messtechnik			Available			
1002	Digitalmultimeter Fluke 87V	Messtechnik			Available			
1003	Digitalmultimeter Fluke 87V	Messtechnik			Available			
1004	Digitalmultimeter Fluke 87V	Messtechnik	Sven Roth	Andreas Hofmann	Overdue	09.09.2026	16.09.2026	BV-2026-0138
1005	Digitalmultimeter Fluke 87V	Messtechnik	Jonas Wagner	Marcel Walter	LentOut	06.10.2026	13.10.2026	BV-2026-0131
1006	Digitalmultimeter Fluke 87V	Messtechnik			Available			
1007	Digitalmultimeter Fluke 87V	Messtechnik			Available			
1008	Digitalmultimeter Fluke 87V	Messtechnik			Available			
1009	Zangenampere meter Fluke 37	Messtechnik			Available			
1010	Zangenampere meter Fluke 37	Messtechnik			Available			
1011	Zangenampere meter Fluke 37	Messtechnik			Available			
1012	Zangenampere meter Fluke 37	Messtechnik	Jonas Wagner	Alexander Frank	LentOut	03.10.2026	10.10.2026	BV-2026-0124
1013	Zangenampere meter Fluke 37	Messtechnik			Available			
1014	Zangenampere meter Fluke 37	Messtechnik	Jonas Wagner	Martin Seidel	LentOut	04.10.2026	11.10.2026	BV-2026-0124
1015	Isolationsschere Fluke 15C	Messtechnik	Sven Roth	Timo Berger	Overdue	11.09.2026	14.09.2026	BV-2026-0138
1016	Isolationsschere Fluke 15C	Messtechnik			Available			
1017	Isolationsschere Fluke 15C	Messtechnik			Available			
1018	Isolationsschere Fluke 15C	Messtechnik			Retired			
1019	Isolationsschere Fluke 15C	Messtechnik	Sven Roth	Christian Hartmann	LentOut	07.10.2026	10.10.2026	BV-2026-0145
1020	Spannungsprüfer Benning Du	Messtechnik			Available			
1021	Spannungsprüfer Benning Du	Messtechnik			Available			
1022	Spannungsprüfer Benning Du	Messtechnik			Available			
1023	Spannungsprüfer Benning Du	Messtechnik			Available			
1024	Spannungsprüfer Benning Du	Messtechnik			Available			
1025	Spannungsprüfer Benning Du	Messtechnik			Available			
1026	Netzanalysator Fluke 1748	Messtechnik			Available			
1027	Netzanalysator Fluke 1748	Messtechnik			Available			
1028	Netzanalysator Fluke 1748	Messtechnik			Available			
1029	Installationstester Proffest M	Prüftechnik	Sven Roth	Patrick Neumann	LentOut	07.10.2026	12.10.2026	BV-2026-0138
1030	Installationstester Proffest M	Prüftechnik	Jonas Wagner	Benjamin Herrmann	LentOut	07.10.2026	12.10.2026	WA-2026-0063
1031	Installationstester Proffest M	Prüftechnik	Jonas Wagner	Lukas Hoffmann	LentOut	07.10.2026	12.10.2026	BV-2026-0152

Abb. 9: Hauptfenster nach dem Programmstart

Gerät suchen

In das Suchfeld über der Tabelle einen Teil der Inventarnummer, der Bezeichnung, der Kategorie oder eines Namens eingeben. Die Tabelle zeigt sofort nur noch passende Geräte. Auch ein Datum im Format TT.MM.JJJJ wird gefunden. Über „Filter zurücksetzen“ erscheinen wieder alle Geräte.

Gerät ausleihen

1. Ein verfügbares Gerät in der Tabelle markieren. Der Button „Neuer Verleih“ ist nur bei verfügbaren Geräten aktiv.
2. „Neuer Verleih“ anklicken.
3. Bei „Entliehen an“ den Monteur und bei „Ausgegeben von“ den Lageristen auswählen. Beide Auswahllisten sind beim Öffnen leer und klappen über den Pfeil auf. „Entliehen an“ enthält alle Mitarbeiter, „Ausgegeben von“ nur das Lagerpersonal. Das Ausgabedatum ist mit dem heutigen Tag vorbelegt.
4. Bei Bedarf ein Rückgabedatum und die Vertragsnummer eintragen. Ohne Rückgabedatum gilt die übliche Leihdauer des Geräts.
5. „Verleihen“ anklicken. Der Button ist erst aktiv, wenn Entleiher, Ausgeber und Ausgabedatum gewählt sind. Bei fehlerhaften Angaben erscheint eine Meldung, und der Dialog bleibt offen.

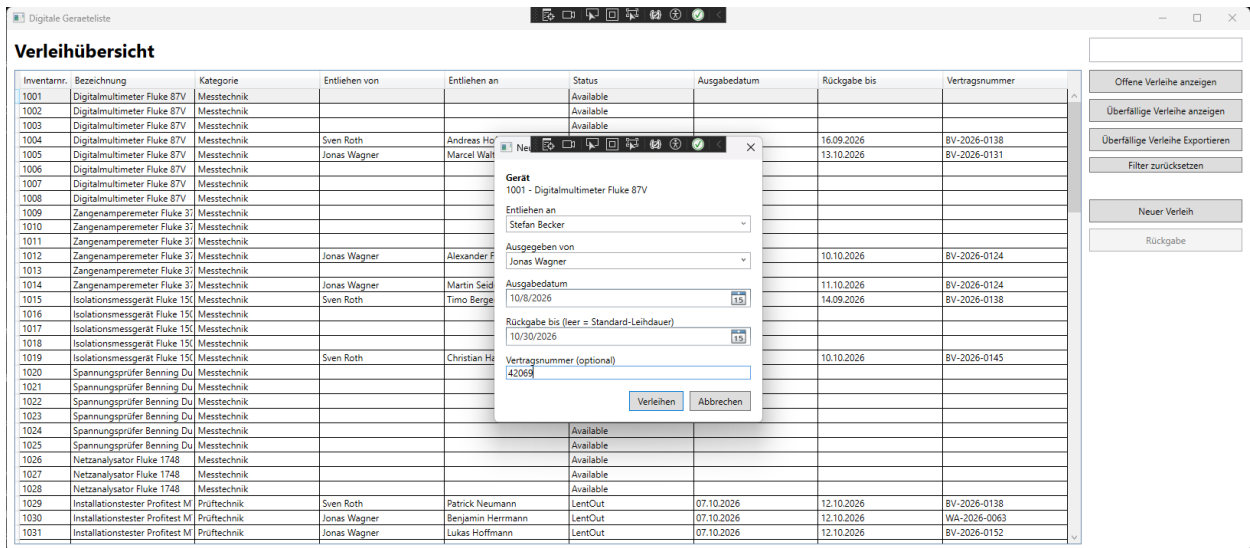


Abb. 10: Ausgefüllter Ausleihdialog mit englischem Datumsformat

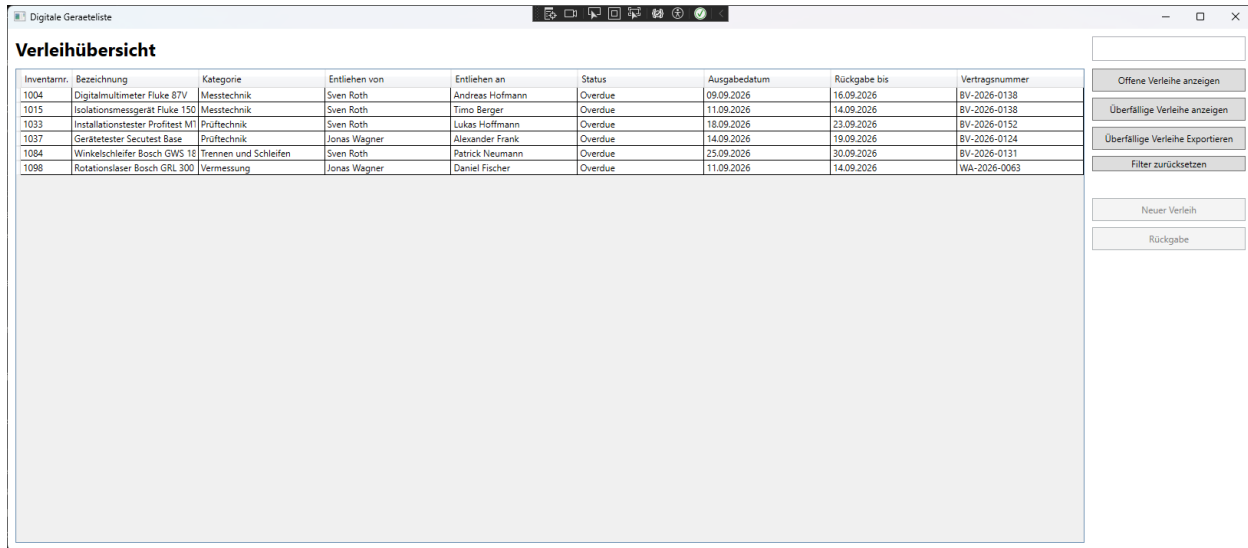
Offene und überfällige Verleihe anzeigen

„Offene Verleihe anzeigen“ listet alle Geräte, die gerade ausgeliehen sind. „Überfällige Verleihe anzeigen“ listet nur Geräte, deren Rückgabedatum überschritten ist. Die Filter wirken nur, solange das Suchfeld leer ist.

The screenshot shows the 'Digitale Geräteliste' application with the 'Verleihübersicht' table. The table is filtered to show only 'Offene Verleihe' (Open Loans). The table columns are: Inventarnr., Bezeichnung, Kategorie, Entliehen von, Entliehen an, Status, Ausgabedatum, Rückgabe bis, and Vertragsnummer. The table lists various equipment items, including digital multimeters, voltage testers, and isolation testers. The 'Status' column shows 'LentOut' for all items. The 'Rückgabe bis' column shows the return date for each item. The 'Vertragsnummer' column shows the contract number for each item.

Inventarnr.	Bezeichnung	Kategorie	Entliehen von	Entliehen an	Status	Ausgabedatum	Rückgabe bis	Vertragsnummer
1001	Digitalmultimeter Fluke 87V	Messtechnik	Jonas Wagner	Stefan Becker	LentOut	08.10.2026	30.10.2026	42069
1004	Digitalmultimeter Fluke 87V	Messtechnik	Sven Roth	Andreas Hofmann	Overdue	09.09.2026	16.09.2026	BV-2026-0138
1005	Digitalmultimeter Fluke 87V	Messtechnik	Jonas Wagner	Marcel Walter	LentOut	06.10.2026	13.10.2026	BV-2026-0131
1012	Zangenampereimeter Fluke 37	Messtechnik	Jonas Wagner	Alexander Frank	LentOut	03.10.2026	10.10.2026	BV-2026-0124
1014	Zangenampereimeter Fluke 37	Messtechnik	Jonas Wagner	Martin Seidel	LentOut	04.10.2026	11.10.2026	BV-2026-0124
1015	Isolationssmessgerät Fluke 150	Messtechnik	Sven Roth	Timo Berger	Overdue	11.09.2026	14.09.2026	BV-2026-0138
1019	Isolationssmessgerät Fluke 150	Messtechnik	Sven Roth	Christian Hartmann	LentOut	07.10.2026	10.10.2026	BV-2026-0145
1029	Installationstester Proffitest M	Prüftechnik	Sven Roth	Patrick Neumann	LentOut	07.10.2026	12.10.2026	BV-2026-0138
1030	Installationstester Proffitest M	Prüftechnik	Jonas Wagner	Benjamin Herrmann	LentOut	07.10.2026	12.10.2026	WA-2026-0063
1031	Installationstester Proffitest M	Prüftechnik	Jonas Wagner	Lukas Hoffmann	LentOut	07.10.2026	12.10.2026	BV-2026-0152
1033	Installationstester Proffitest M	Prüftechnik	Sven Roth	Lukas Hoffmann	Overdue	18.09.2026	23.09.2026	BV-2026-0152
1037	Gerätetester Secutest Base	Prüftechnik	Jonas Wagner	Alexander Frank	Overdue	14.09.2026	19.09.2026	BV-2026-0124
1056	Stemmhammer Bosch GSH 5	Bohren und Stemmen	Sven Roth	Murat Kaya	LentOut	07.10.2026	09.10.2026	BV-2026-0131
1067	Akku-Bohrschrauber Makita D	Schrauben	Jonas Wagner	Dominik Vogel	LentOut	03.10.2026	13.10.2026	BV-2026-0117
1072	Akku-Bohrschrauber Makita D	Schrauben	Sven Roth	Katrin Sommer	LentOut	06.10.2026	16.10.2026	BV-2026-0138
1074	Akku-Bohrschrauber Makita D	Schrauben	Jonas Wagner	Lukas Hoffmann	LentOut	07.10.2026	17.10.2026	WA-2026-0071
1078	Akku-Schlagschrauber Makita	Schrauben	Jonas Wagner	Murat Kaya	LentOut	02.10.2026	12.10.2026	BV-2026-0145
1080	Drehmomentschrauber Wera	Schrauben	Jonas Wagner	Ralf Winkler	LentOut	08.10.2026	15.10.2026	BV-2026-0131
1084	Winkelschleifer Bosch GWS 18	Trennen und Schleifen	Sven Roth	Patrick Neumann	Overdue	25.09.2026	30.09.2026	BV-2026-0131
1089	Akku-Säbelsäge Makita D1R16	Trennen und Schleifen	Jonas Wagner	Dennis Braun	LentOut	06.10.2026	11.10.2026	WA-2026-0063
1098	Rotationslaser Bosch GRL 300	Vermessung	Jonas Wagner	Daniel Fischer	Overdue	11.09.2026	14.09.2026	WA-2026-0063
1118	Leitungssucher Amprobe AT-4	Ortung und Thermografie	Jonas Wagner	Ralf Winkler	LentOut	08.10.2026	11.10.2026	BV-2026-0117

Abb. 11: Filter „Offene Verleihe“



Inventarnr.	Bezeichnung	Kategorie	Entliehen von	Entliehen an	Status	Ausgabedatum	Rückgabe bis	Vertragsnummer
1004	Digitalmultimeter Fluke 87V	Messtechnik	Sven Roth	Andreas Hofmann	Overdue	09.09.2026	16.09.2026	BV-2026-0138
1015	Isolationsmessgerät Fluke 150	Messtechnik	Sven Roth	Timo Berger	Overdue	11.09.2026	14.09.2026	BV-2026-0138
1033	Installationstester Proffitest M1	Prüftechnik	Sven Roth	Lukas Hoffmann	Overdue	18.09.2026	23.09.2026	BV-2026-0152
1037	Gerätetester Secutest Base	Prüftechnik	Jonas Wagner	Alexander Frank	Overdue	14.09.2026	19.09.2026	BV-2026-0124
1084	Winkelschleifer Bosch GWS 16	Trennen und Schleifen	Sven Roth	Patrick Neumann	Overdue	25.09.2026	30.09.2026	BV-2026-0131
1098	Rotationslaser Bosch GRL 300	Vermessung	Jonas Wagner	Daniel Fischer	Overdue	11.09.2026	14.09.2026	WA-2026-0063

Abb. 12: Filter „Überfällige Verleihe“

Gerät zurücknehmen

Das ausgeliehene Gerät markieren und „Rückgabe“ anklicken. Nach der Bestätigung wird die Rückgabe mit dem heutigen Datum gebucht. Das Gerät ist sofort wieder verfügbar.

Überfällige Geräte exportieren

„Überfällige Verleihe Exportieren“ anklicken. Im folgenden Dialog ist der Dateiname mit dem heutigen Datum vorbelegt, zum Beispiel Ueberfaellig_2026-10-08.csv. Speicherort wählen und bestätigen. Gibt es keine überfälligen Geräte, weist die Anwendung darauf hin.

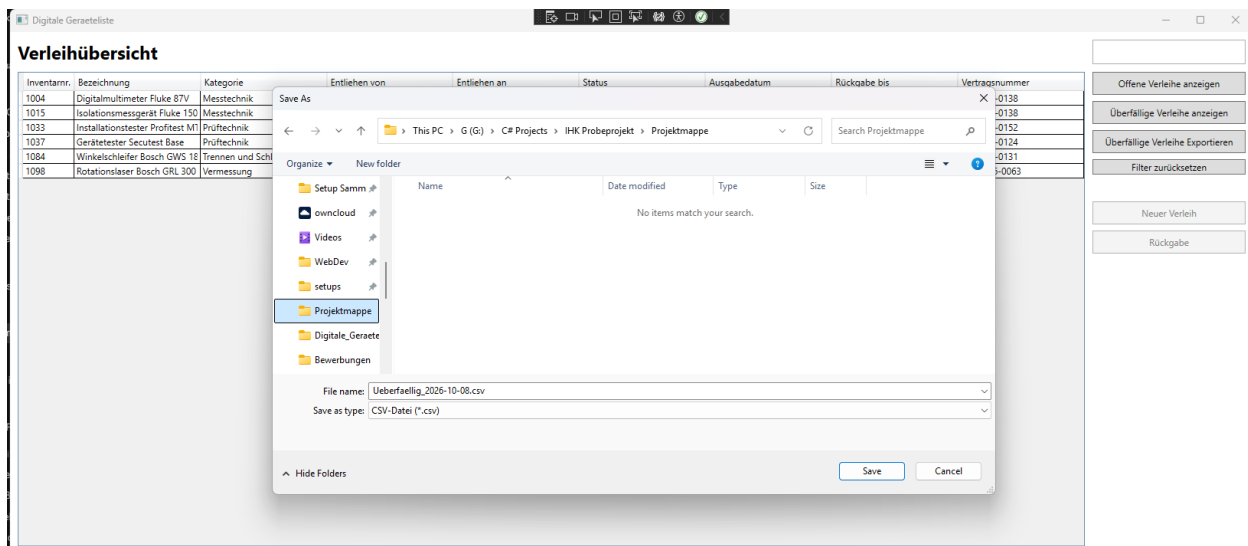


Abb. 13: Speicherort für den Export wählen, Dialog in englischer Systemsprache

Die Datei lässt sich in der deutschen Excel-Version per Doppelklick öffnen und an die Bauleitung weitergeben. Abb. 14 stammt von einem Rechner mit englischer Excel-Version, dort steht jede Zeile in

Spalte A. In einer englischen Version die Datei stattdessen über Daten > Aus Text/CSV mit Semikolon als Trennzeichen öffnen.

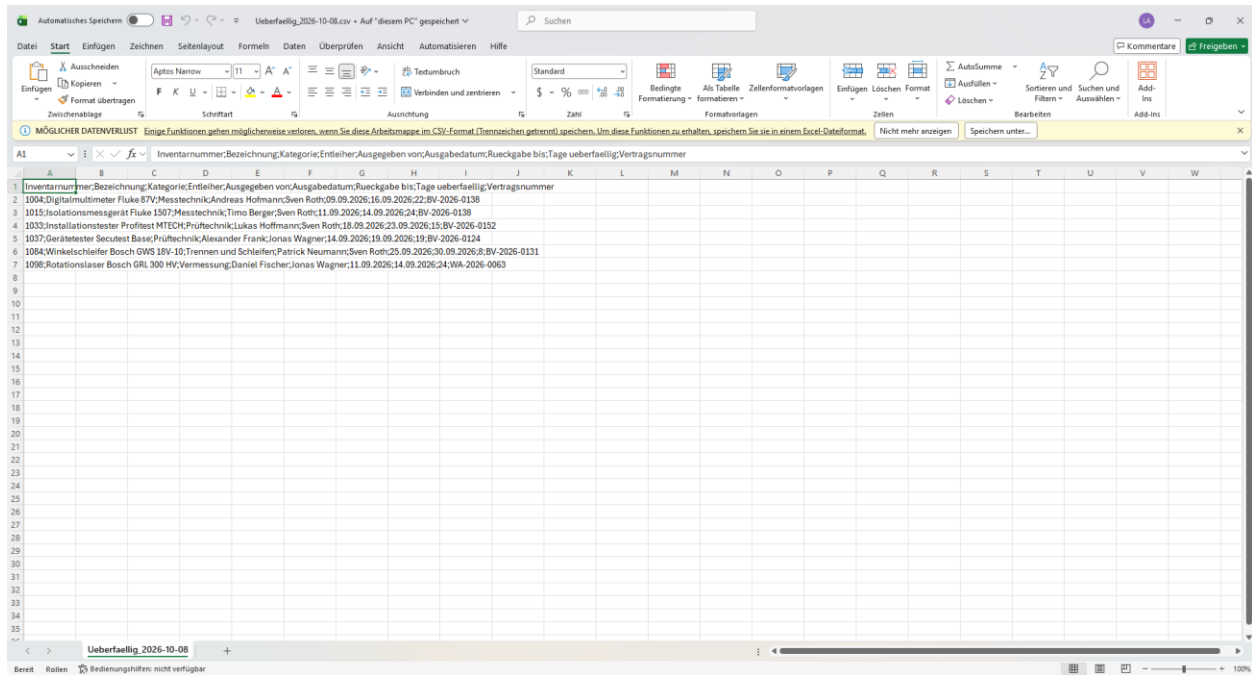


Abb. 14: Exportierte Überfälligkeitsliste in einer englischen Excel-Version

A6 Codeauszüge

Die folgenden Auszüge zeigen die zentralen Stellen der Fachlogik und eine Stelle mit Überarbeitungsbedarf.

```
2 references
public TransactionResult BorrowItem(int itemId, int borrowedById, int lendById, DateTime lendDate, DateTime? expectedReturnDate, string affiliatedContractNumber)
{
    var item = _itemRepository.GetItemById(itemId);
    if (item == null)
    {
        return TransactionResult.Failure("Item not found.");
    }
    if (expectedReturnDate.HasValue && expectedReturnDate.Value.Date < lendDate.Date)
    {
        return TransactionResult.Failure("Invalid return date.");
    }
    if (item.IsRetired)
    {
        return TransactionResult.Failure("Item is retired and cannot be borrowed.");
    }
    LendItem? existingLendItem = _lendItemRepository.GetOpenLendById(itemId);
    if (existingLendItem != null)
    {
        return TransactionResult.Failure("Item is already borrowed.");
    }
    bool success = _lendItemRepository.CreateNewLendItem(itemId, borrowedById, lendById, lendDate, affiliatedContractNumber, expectedReturnDate);
    if (!success)
    {
        return TransactionResult.Failure("Failed to create new lend item.");
    }
    return TransactionResult.Success();
}
```

Abb. 15: Prüfungen vor einer Ausleihe in LendService.BorrowItem

```
public TransactionResult ReturnItem(int lendItemId, DateTime returnDate)
{
    var item = _lendItemRepository.GetLendItemById(lendItemId);
    if (item == null)
    {
        return TransactionResult.Failure("Item not found.");
    }
    if (returnDate.Date < item.LendDate.Date)
    {
        return TransactionResult.Failure("Return date cannot be before the lend date.");
    }
    if (item.ActualReturnDate.HasValue)
    {
        return TransactionResult.Failure("Item has already been returned.");
    }
    bool success = _lendItemRepository.ReturnLendItem(lendItemId, returnDate);
    if (!success)
    {
        return TransactionResult.Failure("Failed to return lend item.");
    }
    return TransactionResult.Success();
}
```

Abb. 16: Prüfungen vor einer Rückgabe in LendService.ReturnItem

```
namespace Digitale_Geraeteliste.Core.Services
{
    11 references
    public class TransactionResult
    {
        3 references
        public bool IsSuccess { get; }
        3 references
        public string ErrorMessage { get; } = string.Empty;

        2 references
        private TransactionResult(bool isSuccess, string errorMessage)
        {
            IsSuccess = isSuccess;
            ErrorMessage = errorMessage;
        }

        2 references
        public static TransactionResult Success()
        {
            return new TransactionResult(true, string.Empty);
        }

        9 references
        public static TransactionResult Failure(string errorMessage)
        {
            return new TransactionResult(false, errorMessage);
        }
    }
}
```

Abb. 17: Ergebnisobjekt TransactionResult

Statusermittlung je Gerät in LendService.DetermineItemStatus:

```
private static ItemStatus DetermineItemStatus(Item item, LendItem? openLend, DateTime
dateToCheck)
{
    if (item.IsRetired)
    {
        return ItemStatus.Retired;
    }
    if (openLend == null)
    {
        return ItemStatus.Available;
    }
    return openLend.IsOverdueAt(dateToCheck) ? ItemStatus.Overdue :
ItemStatus.LentOut;
}
```

Überarbeitungsbedarf in MainViewModel.Refresh

Die Methode Refresh baut die Übersicht nach jeder Änderung von Suchtext oder Statusfilter neu auf (Abb. 18). Sie funktioniert, hat aber Schwächen, die bei der nächsten Überarbeitung behoben werden sollen:

- Ist ein Suchtext eingegeben, wird der Statusfilter nicht angewendet. Beide Filter lassen sich deshalb nicht kombinieren.
- Die vier Zweige des switch wiederholen dieselbe Schleife mit demselben Aufruf von GetInventoryOverview. Nur die Bedingung unterscheidet sich.
- Die Suche vergleicht den Status über ToString(), also mit den englischen Namen des Enums. Die Eingabe „verliehen“ findet deshalb nichts.
- Die Bedingung für die Suche ist lang und schwer lesbar. Auch die Prüfung auf einen leeren Suchtext ließe sich mit `string.IsNullOrEmpty` kürzer schreiben.

```
public void Refresh()
{
    Rows.Clear();
    if (TextBoxFilter != null && TextBoxFilter.Length > 0 && TextBoxFilter.Trim().Length > 0)
    {
        foreach (InventoryOverviewRow row in _lendService.GetInventoryOverview(DateTime.Today))
        {
            if (row.InventoryNumber.Contains(TextBoxFilter, StringComparison.OrdinalIgnoreCase) || row.Name.Contains(TextBoxFilter, StringComparison.OrdinalIgnoreCase) ||
                row.Status.ToString().Contains(TextBoxFilter, StringComparison.OrdinalIgnoreCase) ||
                (row.BorrowedByEmployeeName != null && row.BorrowedByEmployeeName.Contains(TextBoxFilter, StringComparison.OrdinalIgnoreCase)) ||
                (row.LentByEmployeeName != null && row.LentByEmployeeName.Contains(TextBoxFilter, StringComparison.OrdinalIgnoreCase)) ||
                row.CategoryName != null && row.CategoryName.Contains(TextBoxFilter, StringComparison.OrdinalIgnoreCase))
            {
                Rows.Add(row);
            }
        }
    }
    else
    {
        switch (SelectedStatus)
        {
            case ItemStatus.Available:
                foreach (InventoryOverviewRow row in _lendService.GetInventoryOverview(DateTime.Today))
                {
                    if (row.Status == ItemStatus.Available)
                    {
                        Rows.Add(row);
                    }
                }
                break;
            case ItemStatus.LentOut:
                foreach (InventoryOverviewRow row in _lendService.GetInventoryOverview(DateTime.Today))
                {
                    if (row.Status == ItemStatus.LentOut || row.Status == ItemStatus.Overdue)
                    {
                        Rows.Add(row);
                    }
                }
                break;
            case ItemStatus.Overdue:
                foreach (InventoryOverviewRow row in _lendService.GetInventoryOverview(DateTime.Today))
                {
                    if (row.Status == ItemStatus.Overdue)
                    {
                        Rows.Add(row);
                    }
                }
                break;
            default:
                foreach (InventoryOverviewRow row in _lendService.GetInventoryOverview(DateTime.Today))
                {
                    Rows.Add(row);
                }
                break;
        }
    }
}
```

Abb. 18: Methode Refresh im MainViewModel

Geplant ist, die Übersicht einmal zu laden, zuerst nach Status und danach nach Suchtext zu filtern und die Suchbedingung in eine eigene Methode auszulagern. Damit wirken beide Filter zusammen, und jede Bedingung steht nur noch an einer Stelle.

A7 Testprotokoll

Die exportierte Datei vom 08.10.2026 ist in Abb. 14 zu sehen. Sie enthält die sechs überfälligen Ausleihen aus Abb. 12.

Quellenverzeichnis

Internetquellen

- Microsoft Learn: Überblick über Entity Framework Core. <https://learn.microsoft.com/de-de/ef/core/> (abgerufen am 14.09.2026)
- Microsoft Learn: SQLite-Datenbankanbieter – EF Core. <https://learn.microsoft.com/de-de/ef/core/providers/sqlite/> (abgerufen am 14.09.2026)
- Microsoft Learn: Migrationsübersicht – EF Core. <https://learn.microsoft.com/de-de/ef/core/managing-schemas/migrations/> (abgerufen am 14.09.2026)
- Microsoft Learn: Eager Loading verwandter Daten – EF Core. <https://learn.microsoft.com/de-de/ef/core/querying/related-data/eager> (abgerufen am 17.09.2026)
- Microsoft Learn: Kaskadenlöschung – EF Core. <https://learn.microsoft.com/de-de/ef/core/saving/cascade-delete> (abgerufen am 15.09.2026)
- Microsoft Learn: Language Integrated Query (LINQ). <https://learn.microsoft.com/de-de/dotnet/csharp/linq/> (abgerufen am 11.09.2026)
- Microsoft Learn: Übersicht über Datenbindung – WPF. <https://learn.microsoft.com/de-de/dotnet/desktop/wpf/data/> (abgerufen am 04.10.2026)
- Microsoft Learn: ICommand-Schnittstelle. <https://learn.microsoft.com/de-de/dotnet/api/system.windows.input.icommand> (abgerufen am 04.10.2026)
- Microsoft Learn: INotifyPropertyChanged-Schnittstelle. <https://learn.microsoft.com/de-de/dotnet/api/system.componentmodel.inotifypropertychanged> (abgerufen am 03.10.2026)
- Microsoft Learn: Model View ViewModel (MVVM). <https://learn.microsoft.com/de-de/dotnet/architecture/maui/mvvm> (abgerufen am 03.10.2026)
- PlantUML: UML Diagramme erstellen (Gantt Chart, Klassendiagramm). <https://plantuml.com/de/> (abgerufen am 08.10.2026)
- App.Diagrams: Anwendungsfalldiagramm erstellen <https://app.diagrams.net> (abgerufen am 12.09.2026)

Vorlage

Projektdokumentation Fachinformatiker Anwendungsentwicklung (Beispiel, mit 94 Punkten bewertet), PDF. Genutzt als Orientierung für Gliederung und Umfang.

Hilfsmittel und mit KI erstellte Arbeitsunterlagen (Claude, Anthropic; Dateien in der Projektmappe; Zeitraum laut Git-Historie)

- Projektauftrag (Projektauftrag - KI erstellt.md), 10.09.2026
- Vorlagen für die UML-Diagramme (Diagramme - KI erstellt.md), 10.09.2026
- Leitfaden Datenhaltung (Persistenz Leitfaden - KI erstellt.md), 11.09. bis 15.09.2026
- Leitfaden MVVM (MVVM Leitfaden - KI erstellt.md), 13.09. bis 15.09.2026
- Anleitung EF Core und LINQ (EF Core und LINQ Schritt fuer Schritt - KI erstellt.md), 14.09. bis 17.09.2026
- Leitfaden Service und Tests (Service und Tests Leitfaden - KI erstellt.md), 17.09.2026
- Restplan für die Umsetzung (Restplan 2 Tage - KI erstellt.md), 01.10.2026
- Textentwurf Entwurfsentscheidung (Entwurfsentscheidung Rueckgabedatum.md), 03.10.2026
- Testdaten (Data/Testdata/*.csv), erstellt im September 2026, Datumswerte verschoben am 08.10.2026

Hinweis zum Einsatz von KI-Werkzeugen

Im Projekt habe ich das KI-Werkzeug Claude (Anthropic) eingesetzt. Ich lege hier offen, an welchen Stellen das geschehen ist.

Den Programmcode der Anwendung habe ich selbst geschrieben. Das betrifft das Datenmodell, die Repositories, die Fachlogik mit den Regeln R1 bis R5 und die Oberfläche. Die KI habe ich dabei als Lernhilfe genutzt. Sie hat mir Erklärungen und Leitfäden zu Entity Framework Core, LINQ und MVVM gegeben und meinen Zwischenstand geprüft. Die Fehler, die sie dabei gefunden hat, habe ich selbst behoben.

Eine Ausnahme im Code ist die Umstellung von der Leihdauer auf das vereinbarte Rückgabedatum. Diese Änderung hat die KI auf meine Anweisung umgesetzt. Die Testdaten im CSV-Format wurden ebenfalls mit der KI erzeugt. Auch die spätere Verschiebung ihrer Datumswerte auf den Abgabezeitraum hat die KI vorgenommen.

Diese Dokumentation wurde in Teilen von der KI entworfen bzw. umgeschrieben. Die Abbildungen 3 bis 8 hat die KI erstellt. Für die Klassendiagramme (Abb. 6 bis 8) entstand zuerst PlantUML-Code, der wegen Größe und Lesbarkeit durch direkt gezeichnete Bilder ersetzt wurde. Den PlantUML-Code für das Gantt-Diagramm (Abb. 1) hat ebenfalls die KI erstellt. Grundlage waren mein Projektauftrag, mein Quellcode und meine Testergebnisse. Ich habe alle erstellten Inhalte geprüft, mit dem tatsächlichen Projektverlauf abgeglichen und angepasst.

Auch eine Kürzung dieser Dokumentation von 38 auf 34 Seiten hat die KI vorgenommen. Dabei wurden Wiederholungen zusammengefasst, Zwischenüberschriften in Listen umgewandelt und Abbildungen verkleinert. Abb. 19 zeigt die Zusammenfassung der Änderungen, wie die KI sie ausgegeben hat. Nach

Ergänzungen wuchs die Dokumentation trotzdem erneut auf 38 Seiten. Danach fand erneut eine fachliche Prüfung mittels KI statt.

Was gekürzt wurde, ohne Inhalt zu verlieren:

- **Wiederholungen gestrichen:** Die Einleitungen von Kapitel 3, 4, 7 und 8 wiederholten nur die Abschnitte darunter. Die Testbarkeit stand fünfmal da, jetzt in 4.2, 4.6 und 6.1. Die englischen Systemeinstellungen sind jetzt ein Absatz in 5.5 statt zwei in 5.4 und 5.5. In 8.2 fasst ein Punkt die Entwurfsänderungen zusammen und verweist auf 4.3, 4.4 und 5.1, statt sie noch einmal zu begründen.
- **Zwischenüberschriften durch Listen ersetzt:** 1.1.1 bis 1.1.4 sind jetzt ein Absatz. 4.4.1 bis 4.4.6 und 5.6.1 bis 5.6.5 sind Listen mit fettem Stichwort.
- **Code-Ausschnitt in 5.3 gestrichen:** Er stand vollständig in Abb. 14.
- **Abbildungen verkleinert:** Abb. 1 auf 80 %, Abb. 2 auf 75 %, Abb. 4 auf 70 %. Sie bleiben lesbar.
- **Doppelte Aussagen verdichtet:** In 8.1, 8.4 und 8.5 sind sie zusammengezogen, alle Zahlen und Ursachen sind noch drin.
- **Gestrichen, ohne dass Information fehlt:** Den Satz, dass die Monteure ihr Werkzeug vor dem Einsatz abholen, und die Begründung, warum kein zweiter Entwickler eingesetzt wurde. Beides sagte nichts Neues.

Dabei korrigiert:

- **Kapitel 7:** Dort stand, die Anwendung lege die Datenbank beim ersten Start an. Das tut sie nicht, die Datenbank entsteht über die Migrationen. Jetzt steht es in 7.1 korrekt.
- **Kapitel 8:** Dort stand „alle fünf Geschäftsregeln sind umgesetzt“. Das Fazit sagt jetzt richtig: R1, R2, R4 und R5 greifen, R3 ist nur in der Datenschicht vorbereitet.
- **Abb. 1:** Unter dem Bild stand noch eine fremde Zelle „Anwendungsfalldiagramm“, die ist weg.

Abb. 19: Zusammenfassung der KI zur Kürzung der Dokumentation

Eidesstattliche Erklärung

Ich versichere, dass ich die vorliegende Projektdokumentation selbstständig angefertigt und keine anderen als die angegebenen Quellen und Hilfsmittel verwendet habe. Alle wörtlich oder sinngemäß übernommenen Stellen sind als solche gekennzeichnet. Die Arbeit hat in gleicher oder ähnlicher Form keiner anderen Prüfungsbehörde vorgelegen.

08.10.2026, Reutlingen

X

Alexander Lang